Multi-Layered Perceptron (MLP)

Chapter 1 — Complete Study Notes

Perceptron · MLP · Backpropagation · Activations · Losses · Initialization · BatchNorm · Dropout · Optimizers

Contents

1 Why Deep Learning? Where Classical Machine Learning Struggles **6**
1.1 Two Pipelines: Hand-Crafted Features vs. Learned Representations 6
1.2 The Curse of Dimensionality 6
1.3 Performance Keeps Growing With Data 7
1.4 Industry Use Cases 7

2 Introduction to Neural Networks & the Perceptron **8**
2.1 Biological Motivation 8
2.2 McCulloch–Pitts Neuron (1943): The First Mathematical Neuron 8
2.3 Hebb's Rule (1949): The First Learning Idea 9
2.4 From Threshold Logic to a Learning Machine 9
2.5 The Perceptron Model 9
2.6 What Does the Perceptron Learn? 10

3 Perceptron Learning Rule **10**
3.1 AND Gate Example (Rosenblatt) 10
3.2 Step Activation Function 11
3.3 Weight Initialization and Update Rule 11
3.4 Full Training Trace: Learning AND From Scratch 11
3.5 Perceptron Convergence Theorem (Rosenblatt 1962; Novikoff 1962) 12
3.6 New Model: Threshold Function 12

4 Limitations: XOR Problem, ADALINE & MADALINE **12**
4.1 XOR Problem — The Great Limitation 12
4.2 ADALINE (1960) — Bernard Widrow 13
4.3 MADALINE 14
4.4 Perceptron vs. Logistic Regression 14

5 Historical Timeline of Neural Networks **15**

5.1 Why Step Function is a Fatal Problem 15

6 Multi-Layer Perceptron (MLP) Architecture 16

6.1 From Single Neuron to Network 16
6.2 Universal Approximation Theorem 16
6.3 General MLP Equations 16
6.4 Why Non-Linearity Is Essential 17
6.5 Solving XOR With One Hidden Layer 17
6.6 Matrix Form for a Mini-Batch and Shape Bookkeeping 18

7 Activation Functions — Theory, Diagrams & Evolution 19

7.1 Step Function (1957) 19
7.2 Sigmoid / Logistic Function (1980s) 19
7.3 Tanh — Hyperbolic Tangent (1990s) 20
7.4 ReLU — Rectified Linear Unit (2010) 21
7.4.1 Dying ReLU Problem 22
7.5 Leaky ReLU (2013) 22
7.6 Parametric ReLU (PReLU) 23
7.7 ELU — Exponential Linear Unit 23
7.8 GELU — Gaussian Error Linear Unit (2016, used in GPT/BERT) 23
7.9 Activation Function Comparison Summary 24
7.10 Choosing the Output Layer: Activation and Loss Go Together 25

8 Forward Propagation — Theory & Numerical Examples 25

8.1 Concept 25
8.2 Single Neuron Numerical Example (Sigmoid + MSE) 25
8.3 Multi-Layer Forward Pass (Binary Classification, 2 hidden neurons) 26

9 Backpropagation — Theory, Chain Rule & Numerical Examples 26

9.1 Why Backpropagation? 26
9.2 Computational Graphs and Local Gradients 27
9.3 The 8-Step Backpropagation Algorithm 28
9.4 Backpropagation — Single Neuron (MSE + Sigmoid) 28
9.5 Backpropagation — Multi-Layer (MSE + Sigmoid) 29
9.6 Vanishing and Exploding Gradients — The Math 29
9.7 Gradient Checking: Verifying Your Backprop 30

10 Loss Functions & Why BCE + Sigmoid = Clean Gradient 30

10.1 Common Loss Functions 30

10.2 Proof: BCE + Sigmoid ⇒ Gradient = $\hat{y} - y$ 30
10.3 Multi-Layer Backprop with BCE 31
10.4 Where the Losses Come From: Maximum Likelihood 31
10.5 Why Not MSE for Classification? 31
10.6 Robust Regression Losses: MAE and Huber 32
10.7 Practical Details: Numerical Stability and Class Imbalance 32

11 Multi-Class Classification — Softmax & Cross-Entropy **32**
11.1 Network Architecture 32
11.2 Softmax Function 33
11.3 Full Numerical Example 33
11.4 Softmax + CE Gradient 33

12 Key Insights, Summary & Big Picture **34**
12.1 Universal Training Loop 34
12.2 Core Gradient Signals 34
12.3 Backpropagation Key Equations 34
12.4 Evolution of the Field — What Each Era Solved 35

13 Completing Chapter 1: Essential Foundations **35**
13.1 Key Terminology: Epoch, Iteration & Batch 35
13.2 Number of Parameters in a Network 36
13.3 Cross-Entropy Loss — Why Log? Why Negative? 37
13.3.1 Why the Logarithm? 37
13.3.2 Why Negative? 38
13.3.3 Full Binary Cross-Entropy Intuition 38
13.4 Softmax — Why Exponentiate? 38
13.5 Decision Boundary: Perceptron vs. MLP 39

14 Data Preparation, Overfitting & Generalization **39**
14.1 Train, Validation and Test Splits 39
14.2 Feature Scaling (Standardization) 40
14.3 Underfitting, Overfitting and the Bias–Variance Trade-off 40

15 Weight Initialization — Fan-in, Fan-out, Xavier & He **41**
15.1 Why Initialization Matters 41
15.2 The Variance Argument 41
15.3 Xavier (Glorot) and He (Kaiming) Initialization 42

16 Batch Normalization **42**
16.1 The Algorithm . . . 42
16.2 Training vs. Inference . . . 43
16.3 Why It Helps, and Where to Put It . . . 43

17 Dropout **43**

18 Regularization: L2, L1 and Early Stopping **44**
18.1 L2 Regularization (Weight Decay) . . . 44
18.2 L1 Regularization (Sparsity) . . . 45
18.3 Early Stopping . . . 45
18.4 Other Regularizers . . . 45

19 Learning Rate and Learning-Rate Schedules **45**
19.1 How Big Should a Step Be? . . . 45
19.2 Learning-Rate Schedules . . . 46

20 Optimizers: From Gradient Descent to AdamW **46**
20.1 Why Do We Need Optimizers? . . . 46
20.2 Convex vs Non-Convex Landscapes . . . 47
20.3 GD, SGD, and Mini-Batch SGD . . . 47
20.4 How GD and SGD Can Fail Near Saddle Points . . . 48
20.5 Momentum . . . 49
20.6 Nesterov Accelerated Gradient (NAG) . . . 51
20.7 AdaGrad: Per-Parameter Adaptive Learning Rates . . . 52
20.8 RMSProp: Fixing AdaGrad's Dying Learning Rate . . . 55
20.9 Adam: Adaptive Moment Estimation . . . 57
20.10 AdamW: Decoupled Weight Decay . . . 61
20.11 The Full Evolution: Why Each Optimizer Exists . . . 63
20.12 Practical Guidelines: Which Optimizer to Use . . . 64
20.13 Summary: All Formulas in One Place . . . 64

21 Non-Technical Guide: What Every Concept Really Means **65**

22 Evaluating a Classifier **69**
22.1 Confusion Matrix and Threshold Metrics . . . 69
22.2 ROC Curve and AUC . . . 70
22.3 Regression Metrics . . . 70

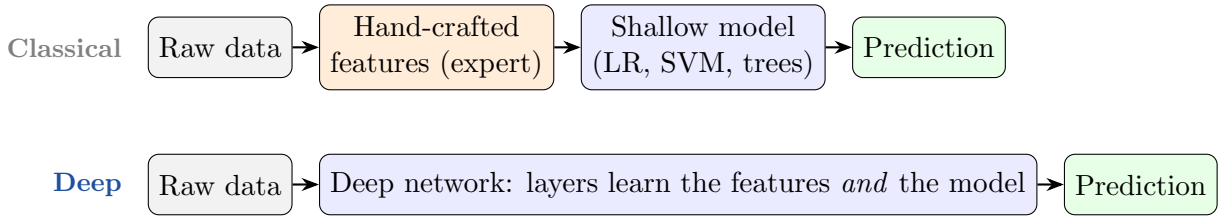
23 Putting It All Together: An MLP in PyTorch **71**

23.1 Practical Checklist 72

1 Why Deep Learning? Where Classical Machine Learning Struggles

Classical machine learning (logistic regression, SVMs, decision trees, gradient boosting) works extremely well when a human can describe the data with a few dozen meaningful columns. It struggles when the raw data is **high-dimensional and unstructured** (pixels, audio samples, words) and the useful features are not known in advance.

1.1 Two Pipelines: Hand-Crafted Features vs. Learned Representations



In the classical pipeline, performance is capped by the quality of the features an expert designs. A deep network learns a **hierarchy of representations**: in images, early layers detect edges, middle layers detect textures and parts, later layers detect objects. This is called **representation learning**.

1.2 The Curse of Dimensionality

Definition: Curse of Dimensionality

As the number of input dimensions d grows, the volume of the input space grows exponentially, so any fixed amount of data covers it ever more sparsely. Methods that rely on "nearby" training examples (k-NN, kernels, shallow models on raw inputs) need exponentially more data.

Three ways to see it:

1. **Coverage.** To sample $[0,1]^d$ on a grid with spacing ε needs $(1/\varepsilon)^d$ points. With $\varepsilon = 0.1$: $d = 2$ needs 100 points, $d = 10$ needs 10^{10}, and a tiny 28×28 image ($d = 784$) needs 10^{784}.
2. **Everything lives near the surface.** The fraction of a unit hypercube's volume within 0.05 of its boundary is

   $$1 - (1 - 2 \times 0.05)^d = 1 - 0.9^d \;=\; 0.19\ (d=2), \quad 0.65\ (d=10), \quad 0.99997\ (d=100).$$

3. **Distances stop being informative.** For random points in high dimensions, the nearest and farthest neighbours are almost the same distance away:

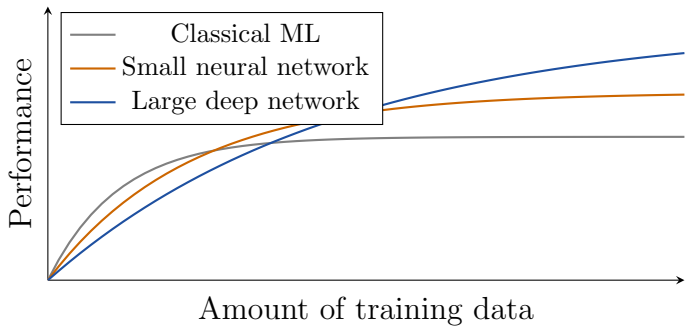
   $$\frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} \longrightarrow 0 \quad \text{as } d \to \infty,$$

   so "similar" examples are hard to find by distance alone.

Deep networks cope because real data is not spread uniformly: images, speech and text lie near much lower-dimensional structures (the *manifold hypothesis*), and layered non-linear features can learn that structure from data.

1.3 Performance Keeps Growing With Data

Schematic: why deep learning took off with big data



On small data, classical models are often as good or better. As data grows, classical models plateau because their capacity is fixed by the features, while larger networks keep improving. Large labelled datasets (ImageNet: 1.2 million images) and GPUs made this regime reachable around 2010–2012.

1.4 Industry Use Cases

Domain	**Raw input**	**Where classical ML struggles**	**What deep learning does**
Medical imaging	CT / X-ray pixels	Hand-designed texture and shape features miss subtle patterns	CNNs learn lesion features directly from labelled scans
Speech	Audio waveform	Hand-tuned acoustic features + HMM pipelines	End-to-end networks map audio to text
Translation, chat	Word sequences	Bag-of-words loses word order and context	Transformers model long-range context
Card fraud	Transaction sequences	Static aggregates miss behavioural sequences	Sequence models and embeddings of merchants, devices
Recommendation	Sparse user / item IDs	Millions of one-hot columns	Learned embeddings for users and items
Self-driving	Camera, LiDAR	Rules cannot enumerate every scene	Perception networks trained on millions of frames

Note

Honest caveat for tabular data. On small or medium spreadsheets (hundreds to tens of thousands of rows, a few dozen columns), gradient-boosted trees and logistic regression are often as accurate as an MLP and easier to tune. Always fit a classical baseline first; use an MLP when you have lots of data, many interacting features, high-cardinality categories, or inputs mixed with text and images.

In Plain Words

A deep network learns the way people build expertise. A radiologist who has read thousands of scans recognises a pattern in seconds, because repetition has strengthened

exactly the right connections. With little experience (small data), a checklist written by an expert (hand-crafted features) does better. With enough experience (big data), learning the patterns directly wins, and keeps improving with more examples.

2 Introduction to Neural Networks & the Perceptron

2.1 Biological Motivation

Neural Networks were inspired by the human brain. The brain consists of billions of **neurons** connected by synapses. Each neuron:

- Receives electrical signals from other neurons.
- Accumulates them (weighted sum).
- **Fires** if the total signal exceeds a threshold.
- Passes the output signal to the next layer of neurons.

Biological neuron ↔ artificial neuron:

Biology	Artificial neuron	Symbol
Dendrites receive signals	Inputs	$x_1, \ldots, x_n$
Synaptic strength	Weights	$w_1, \ldots, w_n$
Cell body integrates charge	Weighted sum	$z = \sum_i w_i x_i + b$
Firing threshold	Bias and activation	b, $F(\cdot)$
Axon carries the output	Output to next layer	$a = F(z)$
Learning strengthens synapses	Training updates weights	$w \leftarrow w - \eta\, \partial \mathcal{L}/\partial w$

Note

The analogy is loose. A human brain has about 86 billion neurons and 10^{14} synapses; biological neurons communicate with timed spikes and learn with local chemical rules. Artificial networks are *inspired* by biology but are, mathematically, compositions of weighted sums and simple non-linear functions.

2.2 McCulloch–Pitts Neuron (1943): The First Mathematical Neuron

Warren McCulloch (neurophysiologist) and Walter Pitts (logician) modelled a neuron as a **threshold logic unit** with binary inputs and output:

$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases} \qquad x_i, y \in \{0, 1\}$$

With all weights $w_i = 1$, the threshold alone decides the logic gate:

x_1	x_2	$x_1 + x_2$	AND ($\theta = 2$)	OR ($\theta = 1$)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	2	1	1

They showed that networks of such units can compute any logical function. The limitation: **weights and thresholds were set by hand**; there was no learning.

2.3 Hebb's Rule (1949): The First Learning Idea

Psychologist Donald Hebb proposed that a connection strengthens when the two neurons it joins are active together — "cells that fire together wire together":

$$\Delta w_i = \eta \, x_i \, y$$

Hebb's rule only ever strengthens co-active connections and has no notion of error, but it introduced the key idea that **weights should be learned from experience**. Rosenblatt's perceptron added the missing ingredient: update weights in proportion to the *error* $y - \hat{y}$.

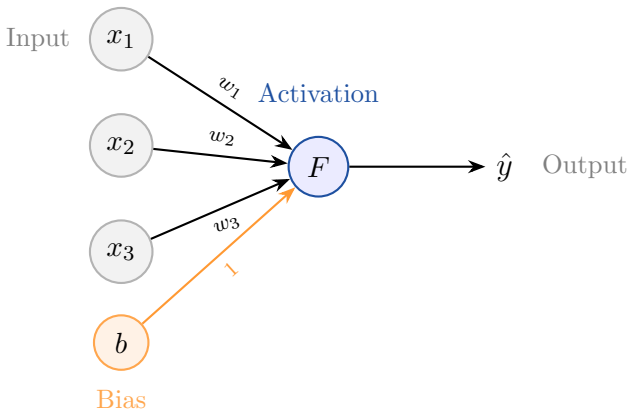
2.4 From Threshold Logic to a Learning Machine

The **Perceptron** (Frank Rosenblatt, 1957–58) combined the McCulloch–Pitts neuron with an error-driven learning rule: the first neuron model that *learns* its weights from labelled examples.

Definition: Perceptron

A single artificial neuron that takes weighted inputs, computes a weighted sum, applies an activation function F, and outputs a binary classification decision. It is the fundamental building block of all neural networks.

2.5 The Perceptron Model



Mathematical formulation:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$$

$$\hat{y} = F(z)$$

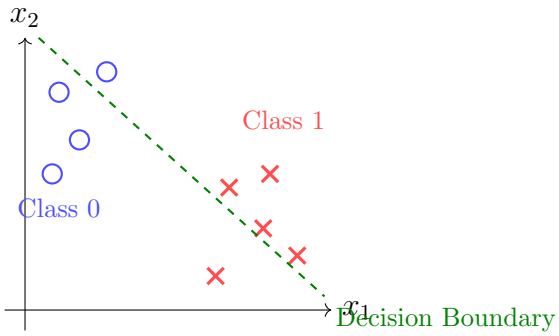
where F is the **activation function**, $\mathbf{w}$ are the learnable weights, and b is the bias term.

Conceptual pipeline:

$$\underbrace{\mathbf{x}}_{\text{Inputs}} \xrightarrow{\times \mathbf{w}} \underbrace{z}_{\text{Weighted Sum}} \xrightarrow{F(\cdot)} \underbrace{\hat{y}}_{\text{Output}}$$

2.6 What Does the Perceptron Learn?

The perceptron learns a **linear decision boundary** — a hyperplane in input space that separates two classes.



Real-world examples:

- **Spam vs. Non-Spam**: email features → spam/not spam.
- **Disease vs. No Disease**: medical measurements → diagnosis.
- **Fraud vs. Legitimate**: transaction features → fraud/legit.

Pros

✓ Simple and fast to train on linearly separable data.
✓ Interpretable — weights directly encode feature importance.
✓ Guaranteed to converge if data is linearly separable (Perceptron Convergence Theorem).
✓ Computationally very efficient: $O(n)$ per update.

Cons

× Can only solve **linearly separable** problems — completely fails on XOR.
× No hidden layers; cannot learn complex non-linear features.
× Hard step function: zero gradient almost everywhere, blocks backpropagation.
× Cannot produce probability estimates — only hard binary outputs.

3 Perceptron Learning Rule

3.1 AND Gate Example (Rosenblatt)

The AND gate is a classic linearly-separable problem solvable by a perceptron:

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

3.2 Step Activation Function

$$F(z) = \text{Step}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

3.3 Weight Initialization and Update Rule

Initialize: $w_1 = 0,\ w_2 = 0,\ b = 0,\ \eta = 1$ (learning rate).

Perceptron Update Rule

For each misclassified training example (x_i, y_i):

$$w_j' = w_j + \eta\,(y - \hat{y})\,x_j \qquad b' = b + \eta\,(y - \hat{y})$$

If prediction is correct: $y - \hat{y} = 0 \Rightarrow$ no update. Otherwise weights shift toward correct direction.

Numerical Example: For $x_1 = 1, x_2 = 1, y = 1$:

$$\begin{aligned} z &= 0(1) + 0(1) + 0 = 0 \ \Rightarrow\ F(0) = 0 \text{ (misclassified)} \\ w_1' &= 0 + 1(1-0)(1) = 1 \\ w_2' &= 0 + 1(1-0)(1) = 1 \\ b' &= 0 + 1(1-0) = 1 \end{aligned}$$

We keep updating weights until classification is correct. This is the original perceptron learning algorithm.

3.4 Full Training Trace: Learning AND From Scratch

Start from $w_1 = w_2 = b = 0$, $\eta = 1$, $\text{Step}(z) = 1$ if $z \geq 0$. Present the four rows in the order $(0,0), (0,1), (1,0), (1,1)$ and apply the update rule after every mistake:

Epoch	**Mistakes**	**Weights after epoch** (w_1, w_2)	**Bias** b
1	2	$(1, 1)$	0
2	3	$(2, 1)$	-1
3	3	$(2, 1)$	-2
4	2	$(2, 2)$	-2
5	1	$(2, 1)$	-3
6	**0**	$(2, 1)$	-3

Check: $z = 2x_1 + x_2 - 3$ gives $-3, -2, -1, 0$ for the four rows, so only $(1,1)$ fires. The learned line $2x_1 + x_2 = 3$ separates AND. Note that the solution is *not unique*: any line separating the point $(1,1)$ from the other three is a valid answer, and the perceptron stops at the first one it finds.

3.5 Perceptron Convergence Theorem (Rosenblatt 1962; Novikoff 1962)

Perceptron Convergence Theorem

Use labels $y_i \in \{-1, +1\}$ and fold the bias into $\mathbf{w}$. Suppose every input satisfies $\|\mathbf{x}_i\| \leq R$, and the data is **linearly separable with margin** $\gamma > 0$: there is a unit vector $\mathbf{w}^\star$ with $y_i\, \mathbf{w}^{\star\top}\mathbf{x}_i \geq \gamma$ for all i. Then the perceptron makes at most

$$\left(\frac{R}{\gamma}\right)^2$$

mistakes, regardless of the order of the examples, and then stops changing.

What this means in practice:

- If the classes are separable, the perceptron is **guaranteed** to find a separating line in finitely many steps.
- A small margin γ (classes nearly touching) means many more mistakes before convergence.
- If the data is **not** separable (XOR, or any noisy real data), the weights never settle: the algorithm cycles forever. This is the formal reason a single perceptron fails on XOR.

3.6 New Model: Threshold Function

The weighted sum $z = x_1 + x_2 + \ldots$ naturally increases with more active inputs, giving a natural threshold behaviour:

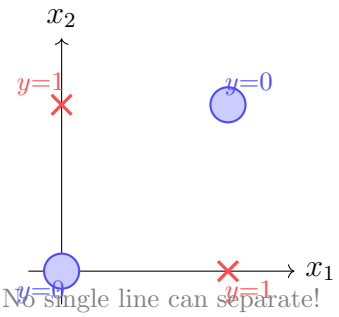
x_1	x_2	$z = x_1 + x_2 + b$
0	0	1
0	1	2
1	0	2
1	1	3

4 Limitations: XOR Problem, ADALINE & MADALINE

4.1 XOR Problem — The Great Limitation

The XOR function cannot be separated by a single line:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Proof: No Single Perceptron Computes XOR

Suppose $\text{Step}(w_1x_1 + w_2x_2 + b)$ computes XOR. The four rows require:

$$\underbrace{b < 0}_{(0,0)\to 0}, \quad \underbrace{w_2 + b \geq 0}_{(0,1)\to 1}, \quad \underbrace{w_1 + b \geq 0}_{(1,0)\to 1}, \quad \underbrace{w_1 + w_2 + b < 0}_{(1,1)\to 0}$$

Add the middle two: $w_1 + w_2 + 2b \geq 0$, so $w_1 + w_2 + b \geq -b > 0$. This contradicts the last inequality. **No choice of weights works.** The same argument applies to a sigmoid neuron thresholded at 0.5, so logistic regression also cannot solve XOR.

This limitation was famously proven by **Marvin Minsky and Seymour Papert** in their 1969 book *Perceptrons*, which effectively killed neural network research funding for over a decade — the first **AI Winter**.

4.2 ADALINE (1960) — Bernard Widrow

ADALINE (**Ada**ptive **Lin**ear **E**lement) replaced the hard step function with a **linear activation**:

$$\hat{y} = F(z) = z = w^T x + b \qquad \text{(identity activation)}$$
$$\text{Training Loss} = \mathcal{L} = (y - \hat{y})^2$$

Weights updated using **gradient descent** (not the hard perceptron rule). This was the first use of continuous gradient-based learning in neural models.

Pros

✓ Continuous loss allows smooth gradient descent.
✓ Better convergence properties than hard perceptron rule.
✓ First model to use MSE and gradient updates.

Cons

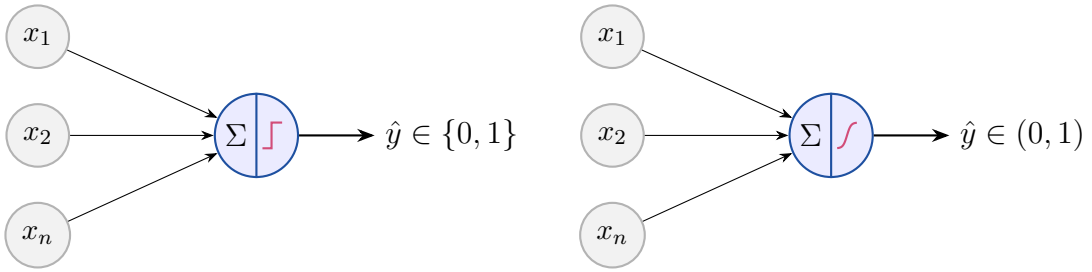
× Still single-layer; cannot solve XOR or non-linear problems.
× Linear activation has very limited representational power.
× No non-linearity ⇒ stacking layers gives no benefit.

4.3 MADALINE

Multiple **ADALINE** neurons combined — the first multi-layer idea. Still lacked a proper algorithm to train hidden layers.

4.4 Perceptron vs. Logistic Regression

Replace the perceptron's hard step with the smooth sigmoid and train with cross-entropy instead of counting mistakes, and you get **logistic regression**. A sigmoid neuron *is* logistic regression, and it is exactly the output unit of a binary-classification MLP.



Two learning rules with the same shape:

$$\text{Perceptron:} \quad \hat{y} = \text{Step}(\mathbf{w}^T\mathbf{x} + b), \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}$$

$$\text{Logistic regression:} \quad \hat{y} = \sigma(\mathbf{w}^T\mathbf{x} + b), \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta\,(\hat{y} - y)\,\mathbf{x}$$

They look identical, but $y - \hat{y} \in \{-1, 0, 1\}$ for the perceptron (it only moves on mistakes), while for logistic regression $\hat{y} - y$ is continuous: it moves on every example, in proportion to how wrong its probability is. The logistic-regression rule is exactly gradient descent on binary cross-entropy (proved in the Loss Functions section: $\partial\mathcal{L}/\partial z = \hat{y} - y$).

	Perceptron	**Logistic regression**
Output	Hard label, 0 or 1	Probability in $(0, 1)$
Activation	Step (gradient 0 almost everywhere)	Sigmoid (smooth)
Loss	Number of mistakes	Binary cross-entropy
Updates	Only on mistakes	On every example
Convergence	Only if linearly separable	Always (convex loss) to the best fit
Decision boundary	A hyperplane	Also a hyperplane (the 0.5 contour)
Can solve XOR?	No	No

In Plain Words

A perceptron is a rubber stamp: approved or rejected, with no sense of how sure it is. Logistic regression is the same machine with a dial that says "73% likely". The dial lets you rank cases, choose a threshold later, and train smoothly. Both still draw only one straight line, so neither can solve XOR. In applied work, always fit logistic

regression first: it is fast, explainable, and the baseline any neural network must beat.

5 Historical Timeline of Neural Networks

1943: McCulloch & Pitts: threshold logic neuron. Weights set by hand.

1949: Hebb: "cells that fire together wire together". First learning idea.

1958: Rosenblatt: the perceptron learns its weights from data (Mark I, 1957–58).

1960: Widrow & Hoff: ADALINE. Linear unit, squared error, gradient descent.

1969: Minsky & Papert, *Perceptrons*: single layers cannot compute XOR.

1970: Linnainmaa: reverse-mode automatic differentiation (the core of backprop).

1974: Werbos: applies the chain rule to train neural networks. Largely unnoticed.

1st AI winter, 1974–1980

1986: Rumelhart, Hinton & Williams: backpropagation trains hidden layers.

2nd AI winter, 1987–1993

1989: Cybenko: universal approximation theorem. LeCun: CNNs read zip codes.

1997: Hochreiter & Schmidhuber: LSTM for long sequences.

2006: Hinton: deep belief nets; layer-wise pre-training revives "deep" learning.

2010: Glorot & Bengio: Xavier initialization. Nair & Hinton: ReLU.

2012: AlexNet wins ImageNet: top-5 error 15.3% vs. 26.2%. Deep learning goes mainstream.

2013: Leaky ReLU (Maas et al.). 2014: Dropout (Srivastava et al.), Adam (Kingma & Ba).

2015: Batch normalization (Ioffe & Szegedy), He initialization, PReLU, ELU, ResNet.

2017: Transformer: "Attention is all you need".

2018: BERT and GPT adopt GELU. 2022: ChatGPT brings LLMs to the public.

5.1 Why Step Function is a Fatal Problem

Deep networks require gradients to train via backpropagation. The step function has:

$$\frac{d}{dz}\text{Step}(z) = \begin{cases} 0 & \text{almost everywhere} \\ \text{undefined} & \text{at } z = 0 \end{cases}$$

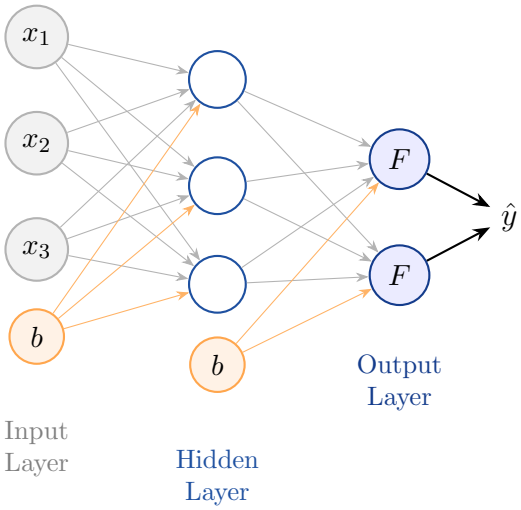
Chain of consequences: No gradient → No backpropagation → Cannot train hidden layers → Cannot go deep

So early neural networks were stuck at one layer.

6 Multi-Layer Perceptron (MLP) Architecture

6.1 From Single Neuron to Network

If a single neuron is powerful, *multiple neurons in layers* can be exponentially more powerful. By stacking layers, the network can learn non-linear, hierarchical representations.



6.2 Universal Approximation Theorem

Universal Approximation Theorem (Cybenko 1989; Hornik 1991; Leshno et al. 1993)

Let F be continuous and not a polynomial (sigmoid, tanh and ReLU all qualify). For any continuous function g on a bounded region $K \subset \mathbb{R}^d$ and any tolerance $\varepsilon > 0$, there is a one-hidden-layer network with some number m of hidden units such that

$$\sup_{\mathbf{x}\in K} \left| g(\mathbf{x}) - \sum_{j=1}^{m} v_j \, F(\mathbf{w}_j^T \mathbf{x} + b_j) \right| < \varepsilon.$$

What the theorem does *not* say: it guarantees that good weights *exist*, not that gradient descent will find them, not how much data is needed, and not how large m must be (it can be astronomically large). In practice, **depth** is far more efficient than width: some functions computed by deep networks need exponentially many units to match with a single hidden layer (Telgarsky, 2016).

6.3 General MLP Equations

For a network with L layers:

$$z^{(l)} = W^{(l)} \, a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = F\big(z^{(l)}\big)$$

where $a^{(0)} = \mathbf{x}$ (inputs), and $\hat{y} = a^{(L)}$ (final output).

The **activation function** F is applied **element-wise** and is the key to non-linearity.

Pros

- ✓ Universal Approximation Theorem: can represent any continuous function.
- ✓ Learns hierarchical features automatically (edges → shapes → objects).
- ✓ Applicable to images, text, tabular data, time series, etc.
- ✓ Scales with data: more data ⇒ better performance.

Cons

- × Requires large amounts of labelled training data.
- × Computationally expensive: many parameters to train.
- × Black-box: hard to interpret what neurons have learned.
- × Prone to overfitting without regularization (dropout, weight decay, etc.).
- × Many hyperparameters: architecture, learning rate, batch size, etc.

6.4 Why Non-Linearity Is Essential

Without an activation function, stacking layers adds nothing. Two linear layers collapse into one:

$$W^{(2)}\left(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)} = \underbrace{W^{(2)}W^{(1)}}_{W'}\,\mathbf{x} + \underbrace{W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}}_{\mathbf{b}'}$$

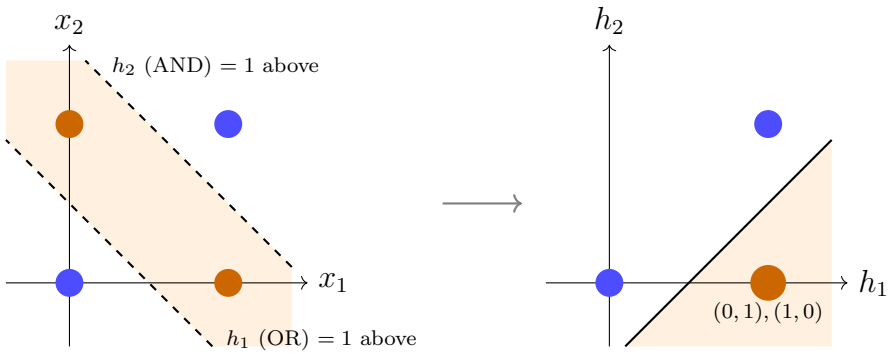
By induction, a 100-layer network with no activations is still a single linear map: it can only draw a straight decision boundary, exactly like one perceptron. The non-linear F between layers is what lets each layer *bend* the space so that later layers can separate what earlier ones could not.

6.5 Solving XOR With One Hidden Layer

Two hidden step units are enough. Let the first detect "at least one input is on" (OR) and the second "both are on" (AND); the output fires for "OR but not AND":

$$h_1 = \text{Step}(x_1 + x_2 - 0.5), \qquad h_2 = \text{Step}(x_1 + x_2 - 1.5), \qquad \hat{y} = \text{Step}(h_1 - h_2 - 0.5)$$

x_1	x_2	h_1 (OR)	h_2 (AND)	$\hat{y}$ (XOR)
0	0	0	0	0
0	1	1	0	**1**
1	0	1	0	**1**
1	1	1	1	0



Input space: two lines **Hidden space: one line**

The hidden layer **re-represents** the data: $(0,1)$ and $(1,0)$ both map to the same hidden point $(1,0)$, and in this new space a single line separates the classes. This is the central idea of deep learning: hidden layers learn features in which the final decision becomes easy. Nobody has to design h_1 and h_2 by hand; with differentiable activations, backpropagation finds such features automatically.

6.6 Matrix Form for a Mini-Batch and Shape Bookkeeping

For one example the layer is $\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$. For a mini-batch of B examples stacked as rows of $A^{(l-1)} \in \mathbb{R}^{B \times n_{l-1}}$, the whole batch is **one matrix multiplication**, which is why GPUs train networks so quickly:

$$Z^{(l)} = A^{(l-1)}\, W^{(l)T} + \mathbf{1}\, \mathbf{b}^{(l)T}, \qquad A^{(l)} = F\big(Z^{(l)}\big), \qquad W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$$

Example: a 5-layer network for heart-disease prediction (13 inputs $\to$ 16 $\to$ 12 $\to$ 8 $\to$ 4 $\to$ 1), batch size $B = 32$:

Layer	Input $A^{(l-1)}$	Weights $W^{(l)}$	Output $A^{(l)}$	Parameters
1 (ReLU)	32×13	16×13	32×16	$13 \cdot 16 + 16 = 224$
2 (ReLU)	32×16	12×16	32×12	$16 \cdot 12 + 12 = 204$
3 (ReLU)	32×12	8×12	32×8	$12 \cdot 8 + 8 = 104$
4 (ReLU)	32×8	4×8	32×4	$8 \cdot 4 + 4 = 36$
5 (sigmoid)	32×4	1×4	32×1	$4 \cdot 1 + 1 = 5$
Total				**573**

Note

Debugging tip: most implementation bugs are shape bugs. Before training, write down the shape of every $A^{(l)}$ and $W^{(l)}$ as above; the inner dimensions of every product must match.

7 Activation Functions — Theory, Diagrams & Evolution

Definition: Activation Function

A (usually non-linear) function $F : \mathbb{R} \to \mathbb{R}$ applied to the weighted sum z of a neuron. It introduces non-linearity into the network, enabling learning of complex patterns. Without activation functions, stacking linear layers is equivalent to a single linear transformation.

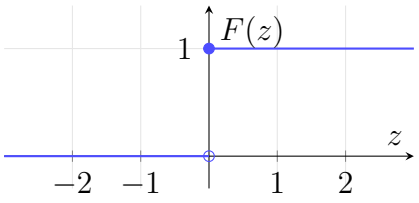
Activation functions evolved to solve three core problems:

1. **Differentiability** — must have usable gradients for backpropagation.
2. **Vanishing Gradients** — gradients must not shrink to zero in deep networks.
3. **Dead Neurons** — neurons must not get permanently stuck at zero output.

7.1 Step Function (1957)

$$F(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases} \qquad F'(z) = \begin{cases} 0 & z \neq 0 \\ \text{undefined} & z = 0 \end{cases}$$

Step Function



Pros

✓ Biologically intuitive: neuron fires or does not fire.
✓ Produces crisp binary outputs.

Cons

× **Zero gradient almost everywhere** ⇒ backpropagation impossible.
× Cannot produce probabilistic outputs.
× Not differentiable at $z = 0$.

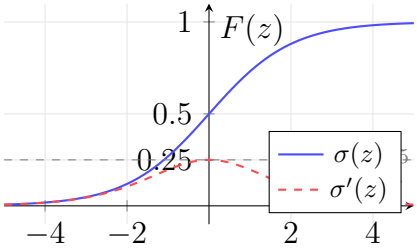
7.2 Sigmoid / Logistic Function (1980s)

The sigmoid function replaced the step function to enable backpropagation:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1), \qquad \sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

Key property: $\sigma'(z)$ maximum value $= 0.25$ (at $z = 0$).

Sigmoid and its Derivative



Vanishing Gradient Problem: When backpropagating through L layers:

$$\text{Gradient} \approx 0.25^L \qquad \Rightarrow \qquad 0.25^{10} \approx 9.5 \times 10^{-7} \approx \mathbf{0.0000009}$$

This gradient is essentially **zero** — the network stops learning in early layers.

Pros

- ✓ Smooth, differentiable everywhere ⇒ enables backpropagation.
- ✓ Output in $(0, 1)$ ⇒ interpretable as probability.
- ✓ Enabled training of multi-layer networks (1986 breakthrough).

Cons

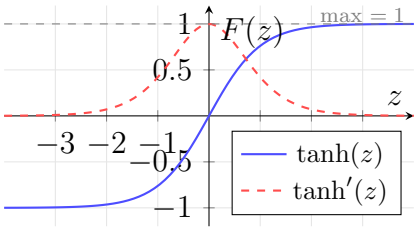
- × **Vanishing gradient:** max derivative = 0.25; shrinks exponentially with depth.
- × **Not zero-centered:** all outputs positive ⇒ inefficient zig-zag gradient updates.
- × Computationally expensive: requires exp(·) and division.
- × Saturates for large $|z|$: gradient ≈ 0 at extremes.

7.3 Tanh — Hyperbolic Tangent (1990s)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, +1), \qquad \tanh'(z) = 1 - \tanh^2(z)$$

Key property: Maximum derivative = 1 (at $z = 0$). **Zero-centered** output.

Tanh and its Derivative



Why Tanh better than Sigmoid?

- Sigmoid output $(0, 1)$ is *not zero-centered*: all gradients for weights in the same layer have the same sign ⇒ inefficient zig-zag updates during gradient descent.
- Tanh output $(-1, +1)$ is *zero-centered*: gradients can be positive or negative ⇒ more efficient updates.

- Max gradient of Tanh = 1 vs. sigmoid's 0.25 ⇒ 4× stronger gradient.

But Tanh still saturates:

$$\text{When } z \to \pm\infty: \ \tanh(z) \to \pm 1 \ \Rightarrow \ \tanh'(z) = 1 - (\pm 1)^2 = 0$$

Vanishing gradient still exists in deep networks.

Pros

✓ Zero-centered: more symmetric, efficient gradient updates.
✓ Max derivative = 1: four times stronger gradient than sigmoid.
✓ Smooth and differentiable everywhere.

Cons

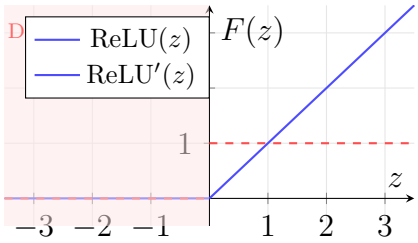
× **Still saturates** at large $|z|$ ⇒ vanishing gradient persists.
× Computationally expensive: two exp calls.
× Deep networks still struggled to train with Tanh.

7.4 ReLU — Rectified Linear Unit (2010)

The **game-changer** for deep learning:

$$\text{ReLU}(z) = F(z) = \max(0, z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases} \qquad F'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

ReLU and its Derivative



Three key advantages of ReLU:

1. No Vanishing Gradient (positive region): Gradient = 1 for $z > 0$ — does not shrink through layers.

2. Computationally Cheap:

$$\text{Sigmoid: requires } \exp(z) \text{ and division} \quad \text{vs.} \quad \text{ReLU: just } \max(0, z)$$

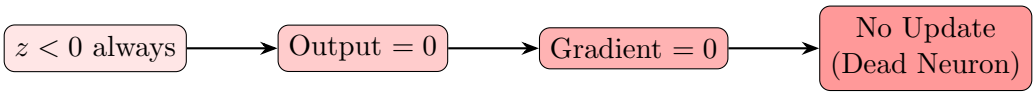
GPU speed improvement enabled much larger, deeper networks.

3. Sparse Activation: For any input, roughly 50% of neurons output 0. This sparsity:

- Reduces memory and computation.
- Acts as implicit regularization (like dropout).
- Creates disentangled, efficient representations.

7.4.1 Dying ReLU Problem

If a neuron receives $z < 0$ consistently, its output $= 0$ *and* gradient $= 0$. The weight update is zero, so the neuron never recovers. It becomes **permanently inactive**.



Pros

- ✓ No vanishing gradient in positive region: gradient $= 1$ always.
- ✓ Computationally very cheap: just $\max(0, z)$.
- ✓ Sparse activation: $\approx 50\%$ neurons inactive $\Rightarrow$ efficiency + regularization.
- ✓ Enabled training of very deep CNNs (AlexNet, VGG, ResNet).
- ✓ Simple and highly effective in practice.

Cons

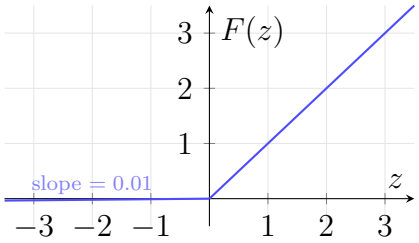
- × **Dying ReLU:** neurons with consistently negative z become permanently dead.
- × **Not zero-centered:** outputs only ≥ 0.
- × Unbounded output can cause instability (exploding activations).
- × Undefined gradient at exactly $z = 0$ (minor; handled by convention as 0).

7.5 Leaky ReLU (2013)

Fix for Dying ReLU: allow a small non-zero gradient for $z < 0$:

$$F(z) = \begin{cases} z & z > 0 \\ 0.01z & z \leq 0 \end{cases} \qquad F'(z) = \begin{cases} 1 & z > 0 \\ 0.01 & z \leq 0 \end{cases}$$

Leaky ReLU



Pros

- ✓ Fixes Dying ReLU: small but non-zero gradient for $z < 0$.
- ✓ Computationally cheap — nearly as fast as ReLU.
- ✓ Retains all positive benefits of ReLU.

Cons

- × Slope 0.01 is a fixed hyperparameter — may not be optimal for all tasks.
- × Inconsistent results across different architectures and datasets.

7.6 Parametric ReLU (PReLU)

$$F(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

The slope α is **learned during training** via backpropagation — not fixed.

Pros

✓ Slope adapts to the data: optimal α per layer/neuron.
✓ Outperforms ReLU and Leaky ReLU on image classification benchmarks.

Cons

× Adds one extra learnable parameter per neuron or layer.
× Risk of overfitting on small datasets.

7.7 ELU — Exponential Linear Unit

$$F(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases} \qquad (\alpha > 0, \text{ typically } \alpha = 1)$$

ELU is smooth at $z = 0$ (unlike Leaky ReLU which has a kink), and has a smooth saturation for very negative inputs.

Pros

✓ Smooth everywhere ⇒ better gradient flow than Leaky ReLU.
✓ Negative outputs possible ⇒ closer to zero-mean activations.
✓ Reduces bias shift during training.

Cons

× Computationally more expensive: requires exp for negative inputs.
× Extra hyperparameter α.

7.8 GELU — Gaussian Error Linear Unit (2016, used in GPT/BERT)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

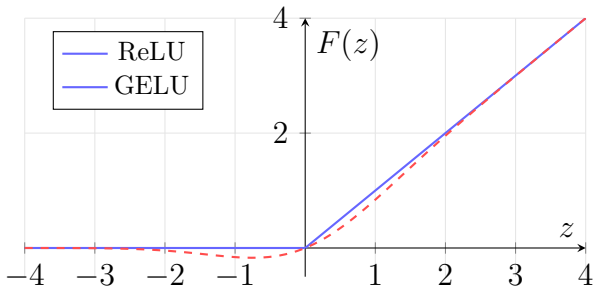
where $\Phi(x)$ is the standard Gaussian CDF: $\Phi(x) = P(X \leq x)$ for $X \sim \mathcal{N}(0, 1)$.

Intuition: GELU *stochastically* gates inputs. It behaves like a soft, smooth version of ReLU — inputs near 0 are gated with probability ≈ 0.5, while large positive inputs are always passed and large negative inputs are always blocked.

A commonly used approximation:

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right]\right)$$

Comparison: ReLU vs. GELU



Pros

- ✓ State-of-the-art in NLP: used in GPT-2, GPT-3, BERT, T5, etc.
- ✓ Smooth everywhere ⇒ no kinks, better optimization landscape.
- ✓ Soft probabilistic gating: more nuanced than hard ReLU threshold.
- ✓ Empirically outperforms ReLU on many NLP and vision benchmarks.

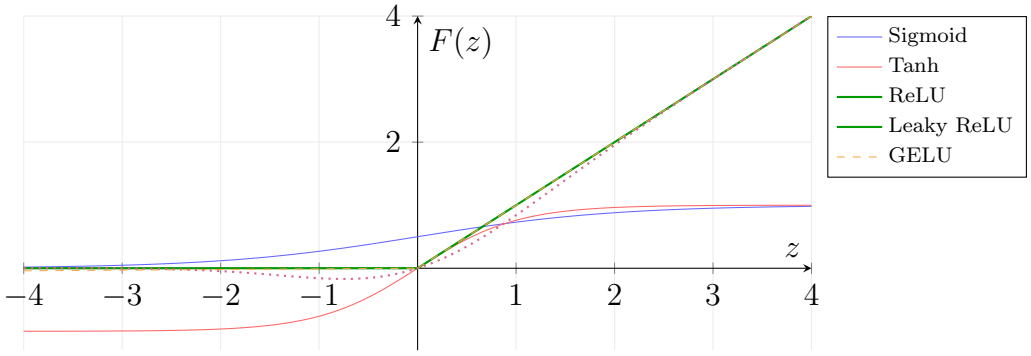
Cons

- × Computationally more expensive than ReLU.
- × Less geometrically interpretable.
- × Not always superior in non-transformer architectures.

7.9 Activation Function Comparison Summary

Function	Range	Zero-Centered	Max Grad	Dying?	Use Case
Step	$\{0, 1\}$	No	0	N/A	Original perceptron
Sigmoid	$(0, 1)$	No	0.25	No	Binary output layer
Tanh	$(-1, 1)$	Yes	1.0	No	Hidden layers (RNNs)
ReLU	$[0, \infty)$	No	1.0	Yes	CNNs, deep nets
Leaky ReLU	$(-\infty, \infty)$	No	1.0	No	General purpose
PReLU	$(-\infty, \infty)$	No	1.0	No	Image classification
ELU	$(-\alpha, \infty)$	Approx	1.0	No	Deep nets
GELU	$\approx (-0.17, \infty)$	Approx	$\approx$1.0	No	Transformers, LLMs

All Activation Functions Compared



7.10 Choosing the Output Layer: Activation and Loss Go Together

Hidden layers almost always use ReLU (or GELU in Transformers). The **output** layer is chosen by the task, and its activation is paired with a matching loss:

Task	**Output units**	**Output activation**	**Loss**
Regression (price, demand)	1 per target	None (linear)	MSE, MAE or Huber
Binary classification (disease, fraud)	1	Sigmoid	Binary cross-entropy
Multi-class, one label (K classes)	K	Softmax	Categorical cross-entropy
Multi-label (several tags at once)	K	Sigmoid on each unit	BCE on each unit

Note

Never put ReLU on the output of a regression model that can predict negative values, and never use softmax for multi-label problems (softmax forces the probabilities to compete and sum to 1).

8 Forward Propagation — Theory & Numerical Examples

8.1 Concept

Forward propagation is the process of passing inputs through the network layer by layer to produce a prediction. At each layer:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = F(z^{(l)})$$

Definition: Forward Pass Pipeline

$$\text{Input } \mathbf{x} \;\rightarrow\; \underbrace{z = \mathbf{w}^T\mathbf{x} + b}_{\text{Linear}} \;\rightarrow\; \underbrace{a = F(z)}_{\text{Activation}} \;\rightarrow\; \underbrace{\mathcal{L}(a, y)}_{\text{Loss}}$$

8.2 Single Neuron Numerical Example (Sigmoid + MSE)

Setup:

$$x_1 = 2,\ x_2 = -1,\ w_1 = 0.3,\ w_2 = -0.2,\ b = 0.05,\ \eta = 0.1,\ y = 1$$

Step 1 — Linear combination:

$$z = x_1 w_1 + x_2 w_2 + b = 2(0.3) + (-1)(-0.2) + 0.05 = 0.6 + 0.2 + 0.05 = \mathbf{0.85}$$

Step 2 — Activation F (Sigmoid):

$$\hat{y} = F(z) = \sigma(0.85) = \frac{1}{1 + e^{-0.85}} \approx \mathbf{0.7006}$$

Step 3 — MSE Loss:

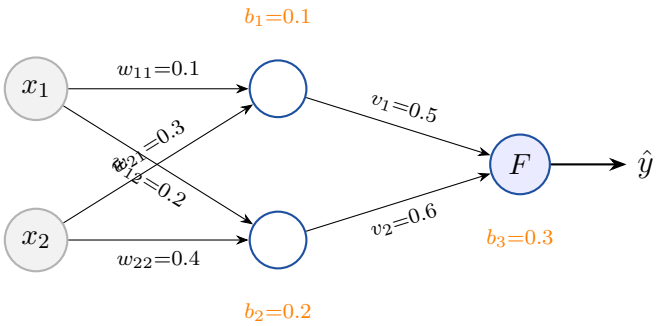
$$L = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1 - 0.7006)^2 = \frac{1}{2}(0.2994)^2 \approx \mathbf{0.0448}$$

Note

Common error: $L = \frac{1}{2}(1 - 0.70)^2 = \frac{1}{2}(0.09) = 0.045$, **not** 0.45. Always multiply by $\frac{1}{2}$.

8.3 Multi-Layer Forward Pass (Binary Classification, 2 hidden neurons)

Network: 2 inputs → 2 hidden neurons → 1 output. Activation: Sigmoid. Loss: MSE.



$$x_1 = 1,\ x_2 = 2,\ y = 1,\ \eta = 0.1$$

Hidden Neuron 1:

$$z_1 = 0.1(1) + 0.3(2) + 0.1 = 0.8, \quad a_1 = \sigma(0.8) \approx 0.68997$$

Hidden Neuron 2:

$$z_2 = 0.2(1) + 0.4(2) + 0.2 = 1.2, \quad a_2 = \sigma(1.2) \approx 0.76852$$

Output Neuron:

$$z_3 = 0.5(0.68997) + 0.6(0.76852) + 0.3 \approx 1.1061, \quad \hat{y} = \sigma(1.1061) \approx 0.75140$$

Loss (MSE):

$$L = \tfrac{1}{2}(1 - 0.75140)^2 \approx \mathbf{0.0309}$$

9 Backpropagation — Theory, Chain Rule & Numerical Examples

9.1 Why Backpropagation?

We need to compute $\dfrac{\partial \mathcal{L}}{\partial w}$ for every weight in the network. Doing this naively is computationally infeasible for deep networks. Backpropagation uses the **chain rule** to compute all gradients in a single backward pass — making training tractable.

The Chain Rule

If $\mathcal{L}$ depends on z through $a = F(z)$, then:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \cdot F'(z)$$

For a full network: $\dfrac{\partial \mathcal{L}}{\partial w} = \dfrac{\partial \mathcal{L}}{\partial a} \cdot \dfrac{\partial a}{\partial z} \cdot \dfrac{\partial z}{\partial w}$

The Deep Insight — Backprop is just repeatedly applying:

$$\text{Gradient}_{\text{current}} = \text{Gradient}_{\text{next}} \times \text{Local Derivative}$$

9.2 Computational Graphs and Local Gradients

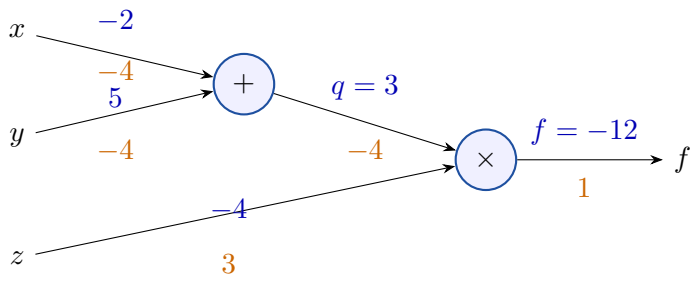
Every calculation a network performs can be drawn as a **computational graph**: nodes are simple operations (add, multiply, activation), edges carry values. Each node only needs to know its own **local derivative**. Backpropagation walks the graph backwards, multiplying the incoming gradient by the local derivative:

$$\underbrace{\frac{\partial f}{\partial \text{input}}}_{\text{downstream gradient}} = \underbrace{\frac{\partial f}{\partial \text{output}}}_{\text{upstream gradient}} \times \underbrace{\frac{\partial \,\text{output}}{\partial \,\text{input}}}_{\text{local gradient}}$$

The four gate rules you need:

Gate	Forward	Local derivatives	Backward behaviour
Add	$q = x + y$	$\frac{\partial q}{\partial x} = 1$, $\frac{\partial q}{\partial y} = 1$	**Distributor**: passes the gradient unchanged to both inputs
Multiply	$f = x\,y$	$\frac{\partial f}{\partial x} = y$, $\frac{\partial f}{\partial y} = x$	**Swapper**: each input gets the gradient times the *other* input
Max (ReLU)	$m = \max(x, y)$	1 for the larger input, 0 for the other	**Router**: sends the whole gradient to the winner
Copy (fan-out)	x used twice	—	**Adder**: gradients from both uses are *summed*

Worked example. $f(x, y, z) = (x+y)\,z$ with $x = -2$, $y = 5$, $z = -4$. Introduce $q = x+y$, so $f = q\,z$.



blue: forward values orange: gradients $\partial f/\partial(\cdot)$

$$\begin{aligned}
\text{Forward:} \quad & q = -2 + 5 = 3, \qquad f = 3 \times (-4) = -12 \\
\text{Start:} \quad & \frac{\partial f}{\partial f} = 1 \\
\text{Multiply gate (swap):} \quad & \frac{\partial f}{\partial z} = q = 3, \qquad \frac{\partial f}{\partial q} = z = -4 \\
\text{Add gate (distribute):} \quad & \frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot 1 = -4, \qquad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot 1 = -4
\end{aligned}$$

Check by intuition: increasing x by a small h raises q by h, and $f = qz$ changes by $z\,h = -4h$. The gradient says exactly that.

Connection to a neuron. A neuron $z = w_1x_1 + w_2x_2 + b$ is a set of multiply gates feeding an add gate. Applying the swap rule gives $\partial z/\partial w_i = x_i$: **the gradient for a weight is the input it multiplies**. This is why every weight gradient in backprop has the form $\delta \cdot a_{\text{input}}$.

9.3 The 8-Step Backpropagation Algorithm

Forward Pass:

1. Inputs $\times$ weights + bias $\Rightarrow z$
2. Apply activation function $F \Rightarrow a$
3. Compute loss $\mathcal{L}(a, y)$

Backward Pass:

4. Compute loss gradient: $\partial\mathcal{L}/\partial\hat{y}$
5. Compute activation gradient: $F'(z)$
6. Combine $\Rightarrow$ error signal $\delta = \frac{\partial\mathcal{L}}{\partial\hat{y}} \cdot F'(z)$
7. Multiply by inputs $\Rightarrow$ weight gradients $\partial\mathcal{L}/\partial w$
8. Update weights: $w \leftarrow w - \eta\,\partial\mathcal{L}/\partial w$

9.4 Backpropagation — Single Neuron (MSE + Sigmoid)

Continuing from Section 8.2:

Step 1 — Loss gradient:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.7006 - 1 = \mathbf{-0.2994}$$

Step 2 — Activation gradient:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) = 0.7006 \times 0.2994 \approx \mathbf{0.2098}$$

$$\frac{\partial L}{\partial z} = (-0.2994)(0.2098) \approx \mathbf{-0.0628} \quad \leftarrow \textit{error signal}$$

Step 3 — Weight gradients:

$$\frac{\partial L}{\partial w_1} = (-0.0628)(2) = \mathbf{-0.1256}, \quad \frac{\partial L}{\partial w_2} = (-0.0628)(-1) = \mathbf{+0.0628}, \quad \frac{\partial L}{\partial b} = \mathbf{-0.0628}$$

Step 4 — Weight updates ($\eta = 0.1$):

$$w_1^* = 0.3 - 0.1(-0.1256) = \mathbf{0.3126}$$
$$w_2^* = -0.2 - 0.1(0.0628) = \mathbf{-0.2063}$$
$$b^* = 0.05 - 0.1(-0.0628) = \mathbf{0.0563}$$

One-line formula (MSE + Sigmoid):

$$\frac{\partial L}{\partial w_i} = (\hat{y} - y) \cdot \sigma'(z) \cdot x_i$$

9.5 Backpropagation — Multi-Layer (MSE + Sigmoid)

Continuing from Section 8.3:

Output error signal:

$$\sigma'(z_3) = 0.75140(1 - 0.75140) \approx 0.18682$$
$$\delta_3 = (\hat{y} - y) \cdot \sigma'(z_3) = (-0.24860)(0.18682) \approx \mathbf{-0.0465}$$

Output weight gradients:

$$dv_1 = \delta_3 \cdot a_1 = (-0.0465)(0.68997) \approx -0.0321, \quad dv_2 = (-0.0465)(0.76852) \approx -0.0357$$

Hidden layer errors:

$$\sigma'(z_1) \approx 0.2139, \quad \sigma'(z_2) \approx 0.1779$$
$$\delta_1 = v_1 \cdot \delta_3 \cdot \sigma'(z_1) = 0.5(-0.0465)(0.2139) \approx \mathbf{-0.00497}$$
$$\delta_2 = v_2 \cdot \delta_3 \cdot \sigma'(z_2) = 0.6(-0.0465)(0.1779) \approx \mathbf{-0.00497}$$

Hidden weight updates ($\eta = 0.1$):

$$v_1^* = \mathbf{0.5032}, \quad v_2^* = \mathbf{0.6036}$$
$$w_{11}^* = \mathbf{0.10050}, \quad w_{21}^* = \mathbf{0.30099}, \quad w_{12}^* = \mathbf{0.20050}, \quad w_{22}^* = \mathbf{0.40099}$$

9.6 Vanishing and Exploding Gradients — The Math

Unrolling the hidden-layer rule $\delta^{(l)} = \left(W^{(l+1)T}\delta^{(l+1)}\right) \odot F'(z^{(l)})$ from the output down to layer 1:

$$\delta^{(1)} = \left(\prod_{l=1}^{L-1} \operatorname{diag}\left(F'(z^{(l)})\right) W^{(l+1)T}\right) \delta^{(L)}, \qquad \left\|\delta^{(1)}\right\| \leq \prod_{l=1}^{L-1} \left(\max|F'| \cdot \|W^{(l+1)}\|\right) \left\|\delta^{(L)}\right\|$$

The gradient reaching the first layer is a **product of $L-1$ factors**. If each factor is below 1 the product shrinks exponentially (**vanishing**); above 1 it grows exponentially (**exploding**).

Per-layer factor	**After 10 layers**	**Effect**
0.25 (sigmoid, $\|W\| \approx 1$)	$0.25^{10} \approx 10^{-6}$	Early layers stop learning
1.0 (ReLU + He init)	1	Healthy gradient flow
1.5 (weights too large)	$1.5^{10} \approx 58$	Huge, unstable updates; NaN losses

Remedies, each covered later in these notes:

- ReLU-family activations ($F' = 1$ when active) instead of sigmoid/tanh in hidden layers.
- Variance-preserving initialization (Xavier, He).
- Batch normalization, which keeps every layer's inputs on a stable scale.
- Gradient clipping for exploding gradients: if $\|g\| > c$, rescale $g \leftarrow c\, g/\|g\|$.
- Residual (skip) connections, $a^{(l+1)} = a^{(l)} + F(\cdot)$, used in ResNets and Transformers.

9.7 Gradient Checking: Verifying Your Backprop

A hand-written backward pass is easy to get subtly wrong. Compare each analytic gradient with a **centred finite difference**:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \epsilon) - \mathcal{L}(\theta_i - \epsilon)}{2\epsilon}, \qquad \epsilon \approx 10^{-5}$$

and report the relative error $\dfrac{|g_{\text{analytic}} - g_{\text{numeric}}|}{\max(|g_{\text{analytic}}|, |g_{\text{numeric}}|)}$:

- $< 10^{-7}$: correct. 10^{-7} to 10^{-4}: suspicious (fine near ReLU kinks). $> 10^{-2}$: there is a bug.

Example. Single neuron from Section 8.2: analytically $\partial L/\partial w_1 = -0.1256$. Nudging $w_1 = 0.3 \pm 10^{-5}$ and recomputing the loss gives the same value to 4 decimal places.

Note

Gradient checking costs two forward passes *per parameter*, so use it only on a tiny network while debugging, then switch it off. It also illustrates why backprop matters: it obtains all gradients for the cost of roughly one extra forward pass.

10 Loss Functions & Why BCE + Sigmoid = Clean Gradient

10.1 Common Loss Functions

Task	Loss	Formula	Gradient
Regression	MSE	$\frac{1}{2}(y - \hat{y})^2$	$\hat{y} - y$
Binary Classification	BCE	$-[y \log \hat{y} + (1-y)\log(1-\hat{y})]$	$\hat{y} - y$ (with Sigmoid)
Multi-class	Cross-Entropy	$-\sum_i y_i \log P_i$	$P - y$ (with Softmax)

10.2 Proof: BCE + Sigmoid $\Rightarrow$ Gradient $= \hat{y} - y$

$$L = -[y \log \hat{y} + (1-y)\log(1-\hat{y})], \qquad \hat{y} = \sigma(z)$$

Using chain rule: $\dfrac{\partial L}{\partial z} = \dfrac{\partial L}{\partial \hat{y}} \cdot \dfrac{\partial \hat{y}}{\partial z}$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}, \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$$

$$\begin{aligned}\frac{\partial L}{\partial z} &= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right)\hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} \\ &= -y + y\hat{y} + \hat{y} - y\hat{y} \\ &= \hat{y} - y\end{aligned}$$

$$\boxed{\frac{\partial L}{\partial z} = \hat{y} - y}$$

The sigmoid derivative *cancels out completely* with the BCE derivative. This is why modern networks always use paired functions: **Sigmoid+BCE** and **Softmax+Cross-Entropy** — the gradients are simple, stable, and easy to implement.

10.3 Multi-Layer Backprop with BCE

Using the same network but with BCE loss:

$$\delta_3 = \hat{y} - y = 0.75140 - 1 = \mathbf{-0.24860} \quad \text{(no sigmoid derivative here — it cancels)}$$

Output weight gradients:

$$dv_1 = (-0.24860)(0.68997) \approx -0.1715, \quad dv_2 = (-0.24860)(0.76852) \approx -0.1911$$

Hidden errors (sigmoid derivative *still applies* at hidden layers):

$$\delta_1 = v_1 \cdot \delta_3 \cdot \sigma'(z_1) = 0.5(-0.24860)(0.2139) \approx -0.0266$$
$$\delta_2 = v_2 \cdot \delta_3 \cdot \sigma'(z_2) = 0.6(-0.24860)(0.1779) \approx -0.0265$$

Weight updates ($\eta = 0.1$):

$$v_1^* = 0.5 - 0.1(-0.1715) = \mathbf{0.5172}, \quad v_2^* = 0.6 - 0.1(-0.1911) = \mathbf{0.6191}$$

10.4 Where the Losses Come From: Maximum Likelihood

Both standard losses are negative log-likelihoods. For a binary label, the network's output $\hat{y} = P(y = 1 \mid \mathbf{x})$ defines a Bernoulli distribution:

$$P(y \mid \mathbf{x}) = \hat{y}^{\,y}(1-\hat{y})^{1-y} \quad \Longrightarrow \quad -\log P(y \mid \mathbf{x}) = -\big[y \log \hat{y} + (1-y)\log(1-\hat{y})\big] = \text{BCE}$$

Minimising average BCE is maximising the probability the model assigns to the observed labels. Likewise, MSE is the negative log-likelihood of a Gaussian noise model $y = \hat{y} + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

10.5 Why Not MSE for Classification?

With a sigmoid output and MSE, the sigmoid derivative does **not** cancel:

$$\frac{\partial L_{\text{MSE}}}{\partial z} = (\hat{y} - y)\,\hat{y}(1-\hat{y})$$

If the model is confidently wrong ($y = 1$, $\hat{y} = 0.01$), this is $(-0.99)(0.01)(0.99) \approx -0.0098$: almost no learning signal exactly when the model most needs to change. With BCE the gradient is $\hat{y} - y = -0.99$, a hundred times larger.

10.6 Robust Regression Losses: MAE and Huber

Loss	Formula	Gradient w.r.t. $\hat{y}$	When to use
MSE	$\frac{1}{2}(y-\hat{y})^2$	$\hat{y}-y$	Default; punishes large errors heavily
MAE	$\lvert y-\hat{y}\rvert$	$\operatorname{sign}(\hat{y}-y)$	Data with outliers; predicts the median
Huber (δ)	$\begin{cases}\frac{1}{2}r^2 & \lvert r\rvert \le \delta \\ \delta(\lvert r\rvert - \frac{1}{2}\delta) & \lvert r\rvert > \delta\end{cases}$	r clipped to $[-\delta,\delta]$	Best of both: smooth near 0, robust to outliers

where $r = \hat{y} - y$.

10.7 Practical Details: Numerical Stability and Class Imbalance

Compute the loss from the logit, not from $\hat{y}$. If z is very negative, $\sigma(z)$ underflows to exactly 0 and $\log 0 = -\infty$. Frameworks combine sigmoid and BCE into one stable formula (PyTorch `BCEWithLogitsLoss`):

$$L(z,y) = \max(z,0) - z\,y + \log\bigl(1+e^{-|z|}\bigr)$$

Similarly, softmax is computed as $e^{z_i-\max_j z_j} / \sum_k e^{z_k-\max_j z_j}$ (subtracting the maximum does not change the result but prevents overflow).

Class imbalance. With 1% fraud, a model that always says "not fraud" is 99% accurate and useless. Weight the rare class more heavily:

$$L = -\bigl[\, w_+\, y \log\hat{y} + (1-y)\log(1-\hat{y})\,\bigr], \qquad w_+ \approx \frac{\#\text{negatives}}{\#\text{positives}}$$

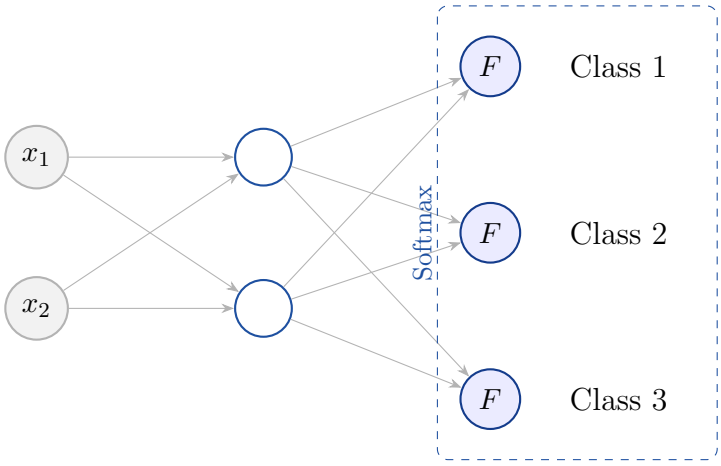
(PyTorch: `BCEWithLogitsLoss(pos_weight=...)`), and evaluate with precision, recall and PR-AUC rather than accuracy (Section 22).

11 Multi-Class Classification — Softmax & Cross-Entropy

11.1 Network Architecture

For K classes, the output layer has K neurons, each computing a logit z_k. A **Softmax** layer converts logits to probabilities.

$$y = [1,0,0] \text{ (True class: Class 1)}, \quad x_1 = 1,\ x_2 = 2$$



11.2 Softmax Function

$$P_i = \text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Properties: $P_i \in (0,1)$ and $\sum_i P_i = 1$ — a valid probability distribution.

11.3 Full Numerical Example

Hidden layer (same as before): $a_1 = 0.68997,\ a_2 = 0.76852$

Output weights: $V = \begin{bmatrix} 0.5 & 0.6 \\ 0.7 & 0.8 \\ 0.9 & 1.0 \end{bmatrix}$, $b^{(o)} = [0.3, 0.4, 0.5]$

Logits:

$$z_1^{(o)} = 0.5(0.68997) + 0.6(0.76852) + 0.3 \approx 1.1061$$
$$z_2^{(o)} = 0.7(0.68997) + 0.8(0.76852) + 0.4 \approx 1.4978$$
$$z_3^{(o)} = 0.9(0.68997) + 1.0(0.76852) + 0.5 \approx 1.8895$$

Softmax probabilities:

$$e^{1.1061} \approx 3.023, \quad e^{1.4978} \approx 4.472, \quad e^{1.8895} \approx 6.615, \quad \text{Sum} = 14.110$$

$$P_1 \approx 0.214, \quad P_2 \approx 0.317, \quad P_3 \approx 0.469$$

Model predicts **Class 3** (highest P). True label is Class 1 $\Rightarrow$ **incorrect**.

Cross-Entropy Loss:

$$L = -\log(P_1) = -\log(0.214) \approx \mathbf{1.541}$$

11.4 Softmax + CE Gradient

$$\delta_i = P_i - y_i: \quad \delta_1 = 0.214 - 1 = -0.786, \quad \delta_2 = 0.317, \quad \delta_3 = 0.469$$

Output weight gradients $\left(\frac{\partial L}{\partial v_{ij}} = \delta_i \cdot a_j\right)$:

$dv_{11} = -0.786 \times 0.68997 \approx -0.5423, \quad dv_{12} = -0.786 \times 0.76852 \approx -0.6043, \quad db_1 = -0.786$

$$dv_{21} = 0.317 \times 0.68997 \approx 0.2187, \quad dv_{22} = 0.317 \times 0.76852 \approx 0.2436, \quad db_2 = 0.317$$
$$dv_{31} = 0.469 \times 0.68997 \approx 0.3236, \quad dv_{32} = 0.469 \times 0.76852 \approx 0.3604, \quad db_3 = 0.469$$

Hidden errors $(V^T \boldsymbol{\delta}) \cdot F'(z)$:

$$\delta h_1 = (0.5(-0.786) + 0.7(0.317) + 0.9(0.469))(0.2139) = (0.2510)(0.2139) \approx \mathbf{0.0537}$$

$$\delta h_2 = (0.6(-0.786) + 0.8(0.317) + 1.0(0.469))(0.1779) = (0.2510)(0.1779) \approx \mathbf{0.0447}$$

Hidden weight gradients:

$$dw_{11} = 0.0537, \; dw_{21} = 0.1074, \quad dw_{12} = 0.0447, \; dw_{22} = 0.0894$$

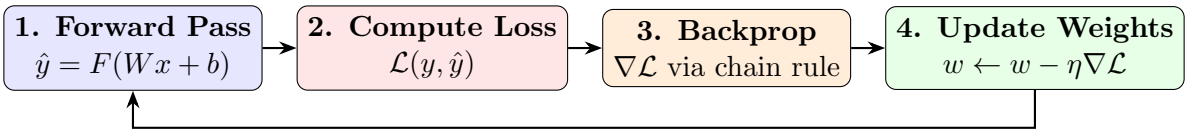
All weight updates $(\eta = 0.1)$**:**

$$w_{11}^* = \mathbf{0.09463}, \; w_{21}^* = \mathbf{0.28926}, \; w_{12}^* = \mathbf{0.19553}, \; w_{22}^* = \mathbf{0.39106}$$
$$v_{11}^* = \mathbf{0.5542}, \; v_{12}^* = \mathbf{0.6604}, \; v_{21}^* = \mathbf{0.6781}, \; v_{22}^* = \mathbf{0.7756}$$
$$v_{31}^* = \mathbf{0.8676}, \; v_{32}^* = \mathbf{0.9640}$$
$$b_1^{(o)*} = \mathbf{0.3786}, \; b_2^{(o)*} = \mathbf{0.3683}, \; b_3^{(o)*} = \mathbf{0.4531}$$

12 Key Insights, Summary & Big Picture

12.1 Universal Training Loop

Every neural network — from a simple perceptron to GPT-4 — uses the same loop:



12.2 Core Gradient Signals

Task	Activation	Loss	Output Gradient δ
Regression	Linear	MSE	$\hat{y} - y$
Binary Classification	Sigmoid	BCE	$\hat{y} - y$
Multi-class	Softmax	Cross-Entropy	$P - y$

Core insight: In all three cases, the gradient is simply **Prediction − Target**. This is why the paired activation+loss combinations are so fundamental.

12.3 Backpropagation Key Equations

$$\delta_{\text{output}} = (\hat{y} - y) \cdot F'(z) \quad \text{(or simply } \hat{y} - y \text{ with BCE/CE)}$$
$$\delta_{\text{hidden}} = (W_{\text{next}}^T \cdot \delta_{\text{next}}) \cdot F'(z_{\text{hidden}})$$
$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \delta_i \cdot a_j, \qquad w^* = w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

12.4 Evolution of the Field — What Each Era Solved

Era	Problem	Solution
1957–1960	Binary classification	Perceptron + Step function
1960–1969	Continuous learning	ADALINE + MSE + Gradient descent
1969	XOR crisis / AI Winter	Recognized limitations; need multi-layer
1986	Cannot train deep nets	Backprop + Sigmoid
1986–2000	Vanishing gradient	Tanh + better init
2006	Deep net revival	Deep Belief Nets (Hinton)
2010	Fast deep training	ReLU + Xavier init + GPUs
2012	Vision breakthrough	AlexNet (deep CNN + ReLU)
2013–2015	Dead neurons, unstable training	Leaky ReLU / PReLU / ELU, Dropout, BatchNorm, Adam
2017+	Language models	Transformers (GELU from BERT/GPT, 2018)

Pros

- ✓ Backpropagation is theoretically elegant: just repeated chain rule.
- ✓ Scales to arbitrary depth and width.
- ✓ One forward + one backward pass per training step: efficient.
- ✓ Exact gradients (not approximations).
- ✓ Works for any differentiable loss and activation.

Cons

- × Vanishing/exploding gradients in very deep networks (mitigated by ReLU, normalization, skip connections).
- × Requires **differentiable** activation functions everywhere.
- × Memory-intensive: must store all intermediate activations for backward pass.
- × Can get stuck in saddle points or poor local minima (less of a problem for very large models).
- × Hyperparameter sensitivity: learning rate, initialization, architecture all matter greatly.

13 Completing Chapter 1: Essential Foundations

13.1 Key Terminology: Epoch, Iteration & Batch

These three terms are among the most commonly confused in deep learning.

Definition: Dataset Terminology

- **Dataset**: The full collection of training examples, e.g. 60,000 images.
- **Batch (Mini-batch)**: A small subset of the dataset processed together, e.g. 32 images. One batch = one gradient update.
- **Iteration**: One forward + backward pass on one batch. After one iteration, weights are updated once.
- **Epoch**: One complete pass through the *entire* training dataset.

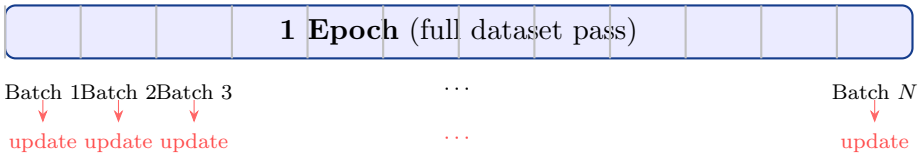
Relationship:

$$\text{Iterations per Epoch} = \frac{\text{Total training examples}}{\text{Batch size}}$$

Example: Dataset = 10,000 samples. Batch size = 100.

$$\text{Iterations per epoch} = \frac{10{,}000}{100} = 100$$

Training for 50 epochs = $100 \times 50 = 5{,}000$ total weight updates.



Term	Size	Gradient	Speed
Batch GD	Full dataset	Exact, smooth	Very slow, stable
Mini-batch GD	32–512 samples	Approximate, noisy	Fast, good balance
SGD	1 sample	Very noisy	Fast but unstable

In practice, **mini-batch gradient descent** (batch size 32–512) is the standard. It balances the stability of full-batch GD with the speed of SGD.

13.2 Number of Parameters in a Network

Understanding how many learnable parameters a network has is fundamental — it determines memory cost, training time, and risk of overfitting.

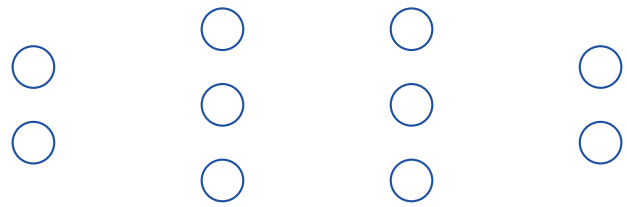
Parameter Count Formula

For a fully-connected layer connecting n_{in} neurons to n_{out} neurons:

$$\text{Parameters} = \underbrace{n_{\text{in}} \times n_{\text{out}}}_{\text{weights}} + \underbrace{n_{\text{out}}}_{\text{biases}}$$

Total parameters in network $= \sum_{\text{all layers}} \left(n_{\text{in}}^{(l)} \times n_{\text{out}}^{(l)} + n_{\text{out}}^{(l)} \right)$

Example: Network with architecture $2 \to 3 \to 3 \to 2$:



Input
2 neurons

Hidden 1
3 neurons

Hidden 2
3 neurons

Output
2 neurons

Layer	n_{in}	n_{out}	**Weights**	**Biases**
Input → Hidden 1	2	3	$2 \times 3 = 6$	3
Hidden 1 → Hidden 2	3	3	$3 \times 3 = 9$	3
Hidden 2 → Output	3	2	$3 \times 2 = 6$	2
Total			**21**	**8**
Grand Total				**29 parameters**

Note

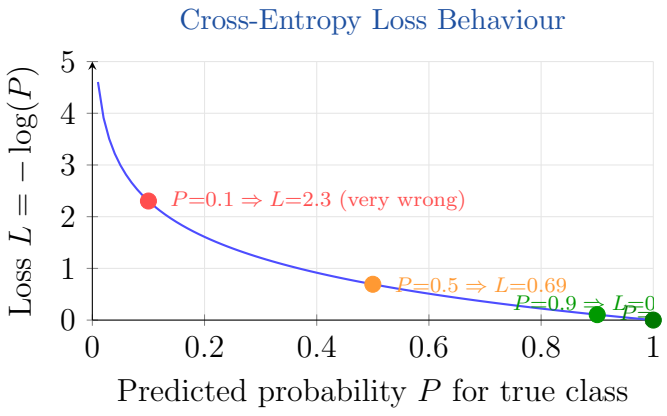
Modern networks like GPT-3 have **175 billion** parameters. GPT-4's size has not been officially disclosed. This is why training them requires thousands of GPUs and months of compute.

13.3 Cross-Entropy Loss — Why Log? Why Negative?

The formula $L = -\log(P_{\text{true}})$ may look arbitrary. Here is the full intuition.

13.3.1 Why the Logarithm?

Consider a model predicting the true class with probability P:



Key behaviours:

- When $P_{\text{true}} \to 1$ (perfect prediction): $-\log(1) = 0$ — **zero loss**.
- When $P_{\text{true}} \to 0$ (completely wrong, confident): $-\log(0) \to \infty$ — **infinite loss**.
- The log function **penalises confident wrong predictions far more heavily** than uncertain ones. This is exactly what we want.

13.3.2 Why Negative?

Since $P \in (0, 1)$, we have $\log(P) \in (-\infty, 0)$ — always negative. The negative sign flips it so that loss is always **positive** and we can **minimise** it:

$$L = -\log(P_{\text{true}}) \geq 0$$

13.3.3 Full Binary Cross-Entropy Intuition

$$L = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big]$$

- When $y = 1$: only $-\log \hat{y}$ matters. Penalises if $\hat{y}$ is small.
- When $y = 0$: only $-\log(1 - \hat{y})$ matters. Penalises if $\hat{y}$ is large.
- **Exactly one term is active per sample.**

13.4 Softmax — Why Exponentiate?

The Softmax function converts raw logits $z_1, z_2, \ldots, z_K$ into probabilities:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Why not just normalise directly (i.e. $P_i = z_i / \sum z_j$)?

1. **Logits can be negative.** Direct normalisation breaks for negative values. $e^z > 0$ always.
2. **Amplifies differences.** Exponentiation makes the largest logit dominate much more strongly, producing sharper, more decisive predictions.
3. **Differentiable.** e^z is smooth everywhere, enabling clean gradients.
4. **Probabilistic interpretation.** Softmax is the natural multi-class generalisation of the sigmoid: with $K = 2$ classes, $P_1 = e^{z_1}/(e^{z_1} + e^{z_2}) = \sigma(z_1 - z_2)$. Logits act as log-odds, and differences between logits are what matter (adding a constant to every z_i changes nothing).

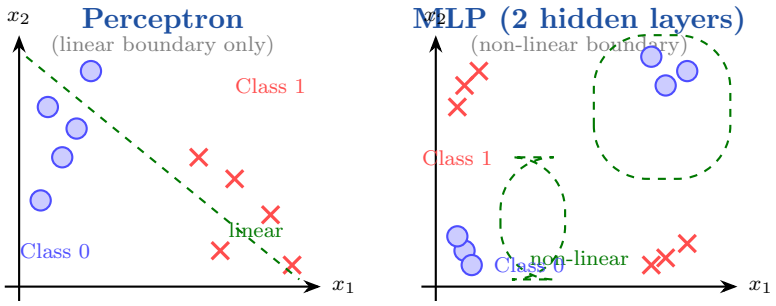
Same logits, two normalisation methods ($z = [2,\ 1,\ -1]$, so $\sum z = 2$):

Class	**Logit** z	**Direct** $z / \sum z$	**Softmax** $e^z / \sum e^z$
Class 1	2.0	100%	70.5%
Class 2	1.0	50%	25.9%
Class 3	-1.0	-50% **(invalid)**	3.5%
Sum	2.0	100% (but not a distribution)	100% ✓

Direct normalisation produces a "probability" above 1 and a negative one as soon as any logit is negative (and divides by zero if the logits sum to 0). Softmax always gives a valid probability distribution.

13.5 Decision Boundary: Perceptron vs. MLP

The most important conceptual leap from perceptron to MLP is the kind of **decision boundary** each can learn.



Model	Boundary	Can Solve
Perceptron (0 hidden layers)	Straight line (hyperplane)	AND, OR — not XOR
MLP (1 hidden layer)	Curved / piecewise	XOR, convex regions
MLP (2+ hidden layers)	Highly complex, arbitrary	Any continuous pattern

Why Hidden Layers Create Non-Linear Boundaries

Each hidden layer applies a non-linear activation F, which *bends and folds* the input space. With hard threshold units, one hidden layer can carve out any convex region (an intersection of half-planes), and two hidden layers can form unions of such regions, i.e. arbitrary shapes. With smooth activations, the Universal Approximation Theorem says even one hidden layer suffices in principle; extra layers make the same boundary far cheaper to represent.

14 Data Preparation, Overfitting & Generalization

14.1 Train, Validation and Test Splits

Definition: The Three Splits

- **Training set** (typically 70–80%): used to compute gradients and update weights.
- **Validation set** (10–15%): used to choose hyperparameters (learning rate, layers, dropout) and for early stopping. Never used for gradient updates.
- **Test set** (10–15%): used **once**, at the very end, to estimate performance on new data.

Training minimises the **empirical risk** on the training data,

$$\hat{R}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i\big),$$

but what we care about is the **expected risk** $R(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x},y)}\big[\mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}), y)\big]$ on future data. The test set estimates it, and the difference $R - \hat{R}$ is the **generalization gap**. Regularization (dropout, weight decay, early stopping) exists to keep this gap small.

Note

When data has a time order (sales, churn, fraud), split by time: train on the past, validate and test on later periods. A random split leaks future information into training.

14.2 Feature Scaling (Standardization)

Convert each input feature to a z-score using statistics of the **training set only**:

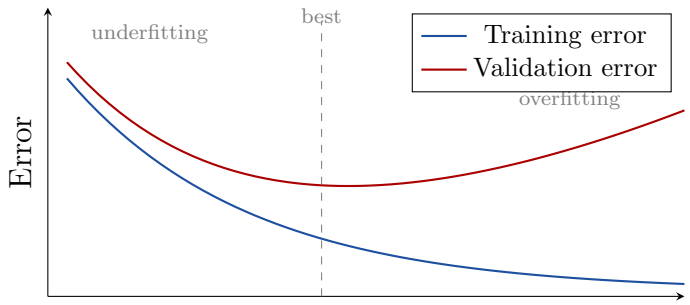
$$x'_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{N}\sum_i x_{ij}, \quad \sigma_j = \sqrt{\frac{1}{N}\sum_i (x_{ij} - \mu_j)^2}$$

Why it matters. If cholesterol ranges in the hundreds and sex is 0/1, the loss surface is a long, narrow valley: one direction is extremely steep, the other almost flat. Gradient descent must use a tiny learning rate for the steep direction and then crawls along the flat one (zig-zagging). For a linear model the curvature matrix is $H = \frac{1}{N}X^T X$; standardizing equalises its diagonal and shrinks its **condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$, turning the valley into a rounder bowl.

Data leakage. Computing μ_j, σ_j on the full dataset lets test-set information shape training. Fit the scaler on the training split, then apply the same stored μ_j, σ_j to validation, test and production data. Categorical features should be one-hot encoded or embedded, not treated as numbers.

14.3 Underfitting, Overfitting and the Bias–Variance Trade-off

Training vs. validation error



Diagnosis	Symptoms	Cause	Fixes
Underfitting (high bias)	Training and validation error both high	Model too simple, or not trained enough	Bigger network, train longer, better features, lower regularization
Overfitting (high variance)	Training error low, validation error rising	Model memorises noise in a small dataset	More data, dropout, weight decay, early stopping, smaller network, augmentation
Good fit	Both low and close together	—	Keep it; check on the test set once

In Plain Words

Training is studying with past exam papers that come with an answer key; the test set is the real exam with new questions. Underfitting is not studying enough. Overfitting is memorising the answer key without understanding the subject: perfect on the practice papers, poor on the exam. Always plot training and validation loss together; the point where validation loss starts rising is where memorisation begins.

15 Weight Initialization — Fan-in, Fan-out, Xavier & He

15.1 Why Initialization Matters

1. Symmetry breaking. If every weight in a layer starts at the same value (for example 0), every neuron in that layer computes the same output, receives the same gradient and makes the same update. They stay identical forever, so a layer of 100 neurons behaves like one neuron. Weights must start **random**. (Biases can safely start at 0.)

2. Signal scale. Each layer multiplies the variance of its signal by some factor. After many layers, a factor slightly above 1 explodes and slightly below 1 vanishes — in the forward activations *and* in the backward gradients.

Definition: Fan-in and Fan-out

For a dense layer $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$: the **fan-in** n_{in} is the number of inputs each neuron receives; the **fan-out** n_{out} is the number of neurons each input feeds. Example: a layer from 256 to 128 units has fan-in 256 and fan-out 128.

15.2 The Variance Argument

Assume weights w_i are independent, zero-mean with variance σ_w^2, and independent of the inputs. For $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$:

$$\text{Var}(z) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(w_i x_i) = n_{\text{in}}\, \sigma_w^2\, \mathbb{E}[x^2]$$

To keep the signal's scale constant from layer to layer we need $n_{\text{in}}\, \sigma_w^2 \approx 1$ (for activations that are roughly linear near zero), or $\frac{1}{2} n_{\text{in}} \sigma_w^2 = 1$ for ReLU, which zeroes about half its inputs so $\mathbb{E}[a^2] = \frac{1}{2}\,\text{Var}(z)$.

What goes wrong otherwise (ReLU, 100 inputs per neuron, 10 layers): with $\sigma_w = 1$ the per-layer factor is $\frac{1}{2} \cdot 100 \cdot 1 = 50$ and the signal grows by $50^{10} \approx 10^{17}$; with $\sigma_w = 0.01$ the factor is 0.005 and the signal shrinks by $\approx 10^{-23}$.

15.3 Xavier (Glorot) and He (Kaiming) Initialization

Initialization Rules

Xavier / Glorot (2010) — for tanh, sigmoid, or linear layers. Balances the forward ($\propto 1/n_{\text{in}}$) and backward ($\propto 1/n_{\text{out}}$) requirements:

$$\sigma_w^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}} \qquad \text{or uniform} \qquad w \sim \mathcal{U}\left[-\sqrt{\tfrac{6}{n_{\text{in}}+n_{\text{out}}}},\ \sqrt{\tfrac{6}{n_{\text{in}}+n_{\text{out}}}}\right]$$

He / Kaiming (2015) — for ReLU and its variants:

$$\sigma_w^2 = \frac{2}{n_{\text{in}}} \qquad \text{or uniform} \qquad w \sim \mathcal{U}\left[-\sqrt{\tfrac{6}{n_{\text{in}}}},\ \sqrt{\tfrac{6}{n_{\text{in}}}}\right]$$

Numerical example (layer $256 \to 128$):

$$\text{Xavier:}\quad \sigma_w = \sqrt{\tfrac{2}{256+128}} = 0.0722, \qquad \text{uniform bound } \sqrt{\tfrac{6}{384}} = 0.125$$

$$\text{He:}\quad \sigma_w = \sqrt{\tfrac{2}{256}} = 0.0884, \qquad \text{uniform bound } \sqrt{\tfrac{6}{256}} = 0.153$$

Activation	**Initialization**	**PyTorch (`nn.init.`)**
ReLU, Leaky ReLU, GELU	He (Kaiming)	`kaiming_normal_(w)`
tanh, sigmoid, linear	Xavier (Glorot)	`xavier_uniform_(w)`
Biases	Zeros	`zeros_(b)`

Note

PyTorch's `nn.Linear` already uses a Kaiming-uniform variant by default, so you rarely set this by hand. Check initialization when the loss does not move at all in the first epoch, or when you build custom layers.

16 Batch Normalization

16.1 The Algorithm

Batch normalization (Ioffe & Szegedy, 2015) standardizes each neuron's pre-activations over the current mini-batch, then lets the network learn the scale and shift it actually wants. For one neuron with values $z_1, \ldots, z_m$ in a batch:

Batch Normalization (training mode)

$$\mu_B = \frac{1}{m}\sum_{k=1}^{m} z_k, \qquad \sigma_B^2 = \frac{1}{m}\sum_{k=1}^{m}(z_k - \mu_B)^2$$

$$\hat{z}_k = \frac{z_k - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_k = \gamma\,\hat{z}_k + \beta$$

γ (scale) and β (shift) are learned by backpropagation, one pair per neuron. $\epsilon \approx 10^{-5}$ avoids division by zero.

Numerical example. One neuron, batch of 4: $z = [2, 4, 6, 8]$, $\gamma = 2$, $\beta = 1$.

$$\mu_B = \tfrac{2+4+6+8}{4} = 5, \qquad \sigma_B^2 = \tfrac{9+1+1+9}{4} = 5$$
$$\hat{z} = \tfrac{z-5}{\sqrt{5}} = [-1.342,\ -0.447,\ 0.447,\ 1.342] \qquad (\text{mean } 0,\ \text{variance } 1)$$
$$y = 2\hat{z} + 1 = [-1.683,\ 0.106,\ 1.894,\ 3.683] \qquad (\text{mean } \beta = 1,\ \text{std } \gamma = 2)$$

16.2 Training vs. Inference

At inference a single patient has no batch, so batch statistics cannot be computed. During training, BN keeps **running averages**:

$$\bar{\mu} \leftarrow \rho\,\bar{\mu} + (1-\rho)\,\mu_B, \qquad \bar{\sigma}^2 \leftarrow \rho\,\bar{\sigma}^2 + (1-\rho)\,\sigma_B^2 \qquad (\rho \approx 0.9)$$

and at test time uses $\hat{z} = (z - \bar{\mu})/\sqrt{\bar{\sigma}^2 + \epsilon}$. In PyTorch this switch is `model.train()` vs. `model.eval()`; forgetting `model.eval()` is a classic cause of "great training accuracy, poor test accuracy".

16.3 Why It Helps, and Where to Put It

- Each layer receives inputs on a stable scale, however earlier layers change (originally explained as reducing "internal covariate shift").
- Later analysis shows it mainly makes the loss surface **smoother** and better conditioned, so **larger learning rates** work and training is less sensitive to initialization (Santurkar et al., 2018).
- Scale invariance: $\mathrm{BN}(\alpha W\mathbf{a}) = \mathrm{BN}(W\mathbf{a})$ for $\alpha > 0$, so growing weights automatically get smaller gradients.
- The mini-batch noise in μ_B, σ_B acts as a mild regularizer.

Typical placement: `Linear` → `BatchNorm` → `ReLU` → `Dropout`. The bias of the Linear layer is then redundant (β replaces it).

Pros

✓ Faster, more stable training; allows higher learning rates.
✓ Less sensitive to weight initialization.
✓ Mild regularization effect.

Cons

× Depends on batch size: unreliable for batches smaller than about 16.
× Different behaviour in training and inference (must call `model.eval()`).
× Awkward for sequences; **Layer Normalization** (normalise across the features of one example) is used in RNNs and Transformers instead.

17 Dropout

Dropout (Srivastava, Hinton et al., 2014) randomly switches off each hidden unit with probability p during training. Every mini-batch therefore trains a different "thinned" sub-network, and all of them share the same weights.

Inverted Dropout (what frameworks implement)

$$m_j \sim \text{Bernoulli}(1-p), \qquad \tilde{a}_j = \frac{m_j\, a_j}{1-p}, \qquad \mathbb{E}[\tilde{a}_j] = a_j$$

Survivors are scaled up by $1/(1-p)$ **during training**, so the expected activation is unchanged. At test time, dropout is switched off and **no scaling is needed**.

Numerical example. $a = [0.8,\ 0.0,\ 1.5,\ 0.4,\ 1.2]$, $p = 0.4$, sampled mask $m = [1, 0, 1, 0, 1]$:

$$\tilde{a} = \frac{[0.8,\ 0,\ 1.5,\ 0,\ 1.2]}{0.6} = [1.333,\ 0,\ 2.5,\ 0,\ 2.0]$$

This particular mask sums to 5.83 instead of 3.9, but averaged over all random masks the expected value of each unit equals its original value.

Why it works:

- **Prevents co-adaptation:** no unit can rely on a specific partner being present.
- **Implicit ensemble:** a layer with n droppable units defines 2^n sub-networks; the full network at test time approximates averaging all of them.
- **Adaptive regularization:** for a linear unit, dropout adds noise with variance $\frac{p}{1-p}\sum_i w_i^2 a_i^2$, which in expectation acts like an L2 penalty weighted by how strongly each input is used.
- **Uncertainty for free (MC dropout):** keep dropout on at test time, run T forward passes, and use the spread of the T predictions as an uncertainty estimate.

Note

Typical p: 0.1–0.5 for hidden layers (0.2–0.3 for small tabular networks); never on the output layer. Switch it off for evaluation with `model.eval()`.

18 Regularization: L2, L1 and Early Stopping

18.1 L2 Regularization (Weight Decay)

Add a penalty on large weights to the loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \frac{\lambda}{2}\sum_j w_j^2 \qquad \Longrightarrow \qquad \frac{\partial \mathcal{L}_{\text{total}}}{\partial w_j} = \frac{\partial \mathcal{L}_{\text{data}}}{\partial w_j} + \lambda\, w_j$$

The SGD update becomes

$$w_j \leftarrow w_j - \eta\Big(\frac{\partial \mathcal{L}_{\text{data}}}{\partial w_j} + \lambda w_j\Big) = \underbrace{(1-\eta\lambda)\, w_j}_{\text{shrink every step}} - \eta\, \frac{\partial \mathcal{L}_{\text{data}}}{\partial w_j}$$

which is why L2 is also called **weight decay**. Example: $w = 0.8$, data gradient 0.3, $\lambda = 0.01$, $\eta = 0.1$ gives $w \leftarrow 0.8 - 0.1(0.3 + 0.008) = 0.7692$. (For Adam, use AdamW, which applies the decay separately; see the AdamW section.)

18.2 L1 Regularization (Sparsity)

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \lambda \sum_j |w_j| \qquad \Longrightarrow \qquad \frac{\partial \mathcal{L}_{\text{total}}}{\partial w_j} = \frac{\partial \mathcal{L}_{\text{data}}}{\partial w_j} + \lambda \, \text{sign}(w_j)$$

The L1 penalty pushes every weight toward zero with a **constant** force λ, however small the weight already is, so weakly useful weights are driven to exactly 0. L2's push (λw_j) fades as the weight shrinks, so weights become small but rarely exactly zero.

	L2 (weight decay)	**L1**
Penalty	$\frac{\lambda}{2}\sum w^2$ (quadratic)	$\lambda \sum \lvert w \rvert$ (linear)
Pull toward zero	Proportional to w	Constant, λ sign(w)
Result	Many small weights	Sparse: many weights exactly 0
Typical use	Default in deep learning	Feature selection, compressed models

18.3 Early Stopping

Monitor validation loss after every epoch, keep the weights from the best epoch, and stop when it has not improved for a set number of epochs (the *patience*, e.g. 10). Early stopping limits how far the weights can travel from their small initial values, which has an effect similar to L2 regularization, and it costs nothing.

18.4 Other Regularizers

Data augmentation (random crops, flips, noise) creates new training examples and is the most effective regularizer for images and audio. **Smaller networks** and **more data** remain the most reliable cures for overfitting.

19 Learning Rate and Learning-Rate Schedules

19.1 How Big Should a Step Be?

The learning rate η is the single most important hyperparameter. A second-order Taylor expansion along the step $-\eta\,\mathbf{g}$ shows the trade-off:

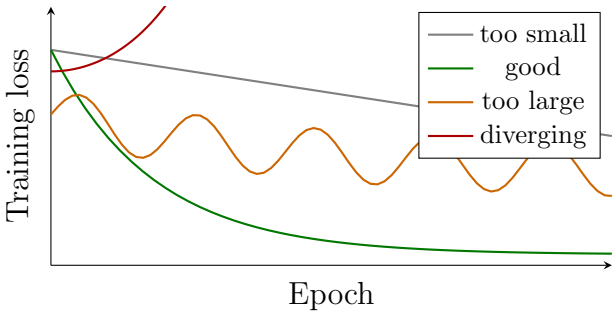
$$\mathcal{L}(\boldsymbol{\theta} - \eta\,\mathbf{g}) \approx \mathcal{L}(\boldsymbol{\theta}) - \eta\,\mathbf{g}^T\mathbf{g} + \frac{\eta^2}{2}\,\mathbf{g}^T H\,\mathbf{g}$$

The first-order term always helps; the curvature term (H = Hessian) hurts when η is too large. On a quadratic bowl, gradient descent is stable only if $\eta < 2/\lambda_{\max}(H)$: the **sharpest** direction sets the speed limit for all others.

Example. $\mathcal{L}(w) = 2w^2$ (so $H = 4$, limit $\eta < 0.5$), start at $w = 1$. Each step multiplies w by $1 - 4\eta$:

η	Factor $1 - 4\eta$	w after 5 steps	Behaviour
0.10	0.6	0.078	Smooth, steady convergence
0.25	0.0	0.000	Optimal: one step
0.45	-0.8	-0.328	Oscillates but converges
0.55	-1.2	-2.488	**Diverges**

Effect of the learning rate (schematic)



19.2 Learning-Rate Schedules

A large rate early makes fast progress; a small rate late settles into a minimum. Schedules change η over training:

Schedule	Formula	Typical use
Step decay	$\eta_t = \eta_0\,\gamma^{\lfloor t/s \rfloor}$ (e.g. $\times 0.1$ every 30 epochs)	Classic CNN training
Exponential	$\eta_t = \eta_0\,e^{-kt}$	Simple smooth decay
Cosine annealing	$\eta_t = \frac{1}{2}\eta_0\big(1+\cos(\pi t/T)\big)$	Transformers, modern default
Linear warm-up	$\eta_t = \eta_0\,t/T_w$ for $t < T_w$, then decay	Large batches, Adam/AdamW
Reduce on plateau	Multiply by 0.1 when validation loss stalls	Practical, data-driven

Cosine example with $\eta_0 = 0.1$, $T = 100$ epochs: $\eta = 0.1,\ 0.085,\ 0.05,\ 0.015,\ 0$ at epochs $0, 25, 50, 75, 100$.

Note

Practical recipe: start from the optimizer's default (10^{-3} for Adam/AdamW, 0.01–0.1 for SGD with momentum). If the loss explodes or oscillates, divide by 3; if it falls very slowly, multiply by 3. Add a schedule once the basic setup trains.

20 Optimizers: From Gradient Descent to AdamW

The sections above explain *what* to compute; optimizers decide *how* to use the gradients to move the weights. Each optimizer below fixes a specific failure of the one before it.

20.1 Why Do We Need Optimizers?

In machine learning, the model predicts

$$\hat{y} = f(x;\, w),$$

where w are trainable weights. We define a loss $L(w)$ that measures how bad the predictions are.

Training means: find weights w that make $L(w)$ as small as possible. We do this iteratively by following the negative gradient direction:

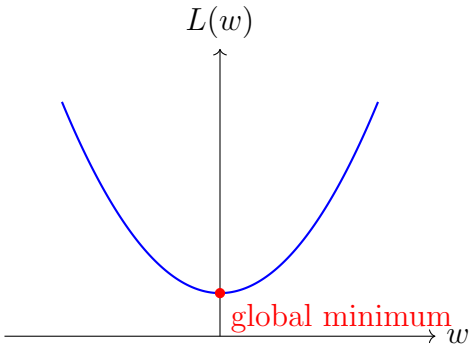
$$w_{t+1} = w_t - \eta \nabla L(w_t),$$

where η is the learning rate.

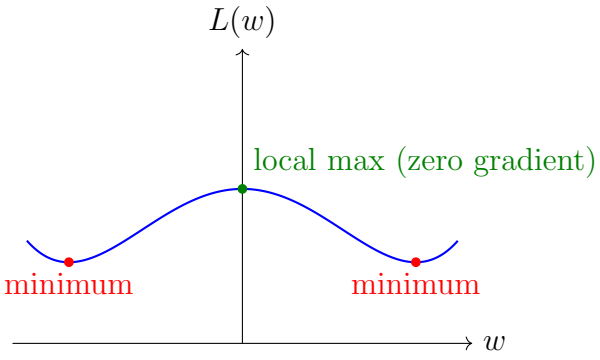
This update rule is exactly why we need optimizers: they tell us *how to compute* and *how to use* gradients to update weights at each step.

20.2 Convex vs Non-Convex Landscapes

Convex: one bowl-like shape — every local minimum is global.



Non-convex: multiple valleys, hills, and flat problematic regions.



In one dimension a zero-gradient point is a minimum or a maximum. In higher dimensions we also get **saddle points**: gradient is zero but the point is not a minimum. Near a saddle, $\nabla L(w) \approx 0$ can make first-order methods appear to stall.

20.3 GD, SGD, and Mini-Batch SGD

Suppose dataset size is N and per-sample loss is $L_i(w)$, with per-sample gradient $g_i = \nabla L_i(w)$.

1) Gradient Descent (GD)

Use all samples every update:

$$g = \frac{1}{N}\sum_{i=1}^{N} g_i, \qquad w \leftarrow w - \eta g.$$

One update per epoch. Stable, but expensive for large N.

Example: $g_1 = 2$, $g_2 = 4$, $g_3 = 6$, $g_4 = 8$:

$$g = \frac{2+4+6+8}{4} = 5, \qquad w \leftarrow w - 0.1 \cdot 5.$$

2) Stochastic Gradient Descent (SGD)

Use one sample every update:

$$g = g_i, \qquad w \leftarrow w - \eta g_i.$$

N updates per epoch. Fast but noisy.

3) Mini-Batch SGD (standard in practice)

Use a small batch of size B:

$$g = \frac{1}{B}\sum_{j=1}^{B} g_j, \qquad w \leftarrow w - \eta g.$$

N/B updates per epoch. Balances noise reduction with computational efficiency.

Method	Data per update	Gradient	Updates/epoch
GD	All N samples	$\frac{1}{N}\sum_{i=1}^{N} g_i$	1
SGD	One sample	g_i	N
Mini-batch SGD	B samples	$\frac{1}{B}\sum_{j=1}^{B} g_j$	N/B

Key Insight

Why mini-batch is the default. Pure GD computes an exact gradient but requires a full pass over data before any weight update, impossibly slow for modern datasets. Pure SGD makes fast updates but the high variance causes zig-zagging that requires very careful learning-rate tuning. Mini-batch SGD occupies the practical sweet spot: enough samples per batch to produce a stable gradient estimate, few enough to update weights many times per epoch and exploit GPU parallelism.

20.4 How GD and SGD Can Fail Near Saddle Points

At a saddle point, $\nabla L(w) = 0$ but the point is not an optimum. Then both GD and SGD produce zero updates:

$$w_{t+1} = w_t - \eta \underbrace{\nabla L(w_t)}_{=0} = w_t.$$

Training stalls. A classic two-variable saddle is $L(w_1, w_2) = w_1^2 - w_2^2$, where the gradient is zero at the origin but it is a minimum along w_1 and a maximum along w_2 simultaneously.

Classifying Critical Points With the Hessian

Near a point where $\nabla L = 0$, the loss is described by its curvature, the Hessian $H = \nabla^2 L$:

$$L(w + \Delta) \approx L(w) + \tfrac{1}{2}\,\Delta^T H\,\Delta, \qquad H = Q\Lambda Q^T$$

The eigenvalues λ_i of H are the curvatures along its eigen-directions:

Eigenvalues of H	**Critical point**	**Example**
All > 0	Local minimum	$w_1^2 + w_2^2$
All < 0	Local maximum	$-w_1^2 - w_2^2$
Mixed signs	**Saddle point**	$w_1^2 - w_2^2$ ($\lambda = 2, -2$)
All ≈ 0	**Plateau** (flat region)	Saturated sigmoids

In a network with millions of parameters, a critical point is a minimum only if *every* eigenvalue is positive, which is unlikely; most high-loss critical points are saddles rather than bad local minima (Dauphin et al., 2014).

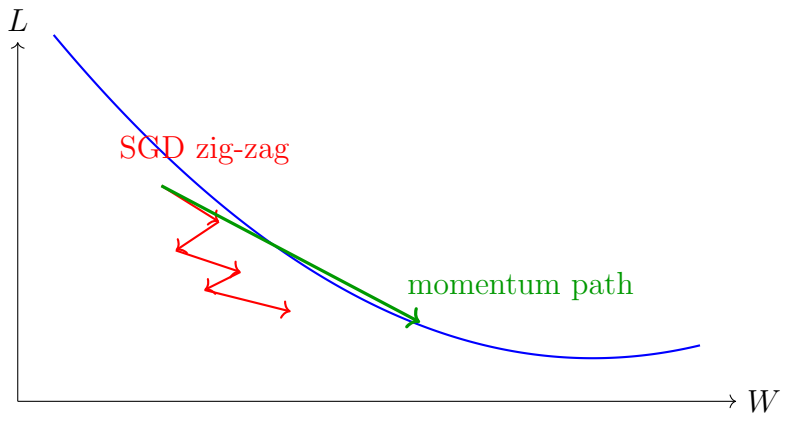
Why plain GD is slow there. Along a direction with $\lambda < 0$, one GD step gives $\Delta_{t+1} = (1 + \eta|\lambda|)\,\Delta_t$, so the escape is exponential, but if the start lies almost exactly on the ridge ($\Delta_0 \approx 0$) it takes many steps before it begins. Three things help: gradient **noise** from mini-batches kicks the weights off the ridge, **momentum** accumulates the small consistent push, and **adaptive methods** (RMSProp, Adam) rescale tiny gradients to a useful step size.

20.5 Momentum

The Problem Momentum Solves: Zig-Zagging

Problem

In narrow loss valleys, plain SGD oscillates back and forth across the steep walls. It makes many costly sideways moves and very little net progress toward the minimum along the valley floor. This is zig-zag behaviour.



Core Idea

Think of a ball rolling down a hill. The gradient gives the local slope. The ball accumulates speed in consistent directions. Sudden direction flips are damped by inertia.

Momentum update rules:

$$v_t = \beta v_{t-1} + (1-\beta)\,\nabla L(W_t)$$

$$W_{t+1} = W_t - \eta\, v_t$$

where v_t is velocity, $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9), and $v_0 = 0$.

Velocity decomposes as:

$$v_t = \underbrace{\beta\, v_{t-1}}_{\text{kept past motion}} + \underbrace{(1-\beta)\,\nabla L(W_t)}_{\text{current push}}.$$

Numerical Example

Numerical Example

Given: $W_0 = 0$, $\eta = 0.1$, $\beta = 0.9$, $v_0 = 0$, gradients $\nabla L(W_0) = 2$, $\nabla L(W_1) = 1.5$, $\nabla L(W_2) = 1$.

Step 1:

$$v_1 = 0.9(0) + 0.1(2) = 0.2, \qquad W_1 = 0 - 0.1(0.2) = -0.02.$$

Step 2:

$$v_2 = 0.9(0.2) + 0.1(1.5) = 0.18 + 0.15 = 0.33, \qquad W_2 = -0.02 - 0.1(0.33) = -0.053.$$

Step 3:

$$v_3 = 0.9(0.33) + 0.1(1) = 0.297 + 0.1 = 0.397, \qquad W_3 = -0.053 - 0.1(0.397) = -0.0927.$$

Velocity grows across steps because the gradient keeps pointing the same direction. Each step is larger and smoother than plain SGD.

Effect of β

- $\beta = 0$: no memory, identical to plain SGD.
- $\beta = 0.9$: strong memory, smooth path, minimal zig-zag.
- $\beta \to 1$: very high inertia, it can overshoot if learning rate is not reduced.

Pros and Cons of Momentum

Advantages	Limitations
Damps zig-zag oscillations	Still uses a fixed global learning rate
Faster convergence in valleys	β needs tuning
Handles noisy gradients better	Can overshoot near sharp minima
No extra memory per parameter	–

20.6 Nesterov Accelerated Gradient (NAG)

The Problem NAG Solves

Problem

Classical momentum evaluates the gradient at the *current* position W_t, before the momentum step has been applied. If velocity is large, the method reacts late and may overshoot a turn or a minimum. It is reactive rather than proactive.

Solution

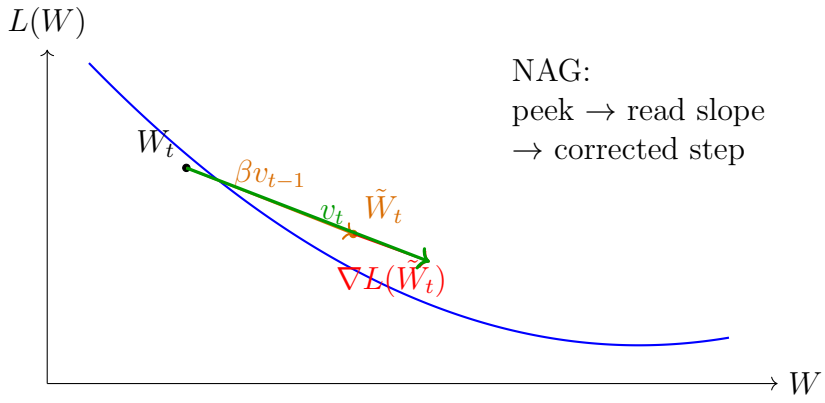
NAG first estimates where momentum is about to carry the weights, then evaluates the gradient *there*. This look-ahead lets the method slow down before overshooting.

Core Formulas

$$\tilde{W}_t = W_t - \beta\, v_{t-1} \quad \text{(look-ahead point)}$$
$$g_t = \nabla L(\tilde{W}_t) \quad \text{(gradient at look-ahead)}$$
$$v_t = \beta\, v_{t-1} + \eta\, g_t$$
$$W_{t+1} = W_t - v_t$$

Important: $\nabla L(\tilde{W}_t)$ means evaluate the loss gradient *at* the look-ahead weight value, not the gradient of the expression $W_t - \beta v_{t-1}$ itself.

Note on conventions: this section writes the velocity with η inside ($v_t = \beta v_{t-1} + \eta g_t$, $W_{t+1} = W_t - v_t$), while the Momentum section used the averaged form $v_t = \beta v_{t-1} + (1-\beta) g_t$, $W_{t+1} = W_t - \eta v_t$. Both are common; they differ only in how η is scaled. PyTorch uses $v_t = \beta v_{t-1} + g_t$, $W_{t+1} = W_t - \eta v_t$.



Numerical Example

Numerical Example

Given: $W_0 = 0$, $v_0 = 0$, $\beta = 0.9$, $\eta = 0.1$, gradient model $\nabla L(W) = 2W + 2$.

Step 1:

$$\tilde{W}_0 = 0 - 0.9(0) = 0, \quad g_0 = 2(0) + 2 = 2,$$

$$v_1 = 0.9(0) + 0.1(2) = 0.2, \quad W_1 = 0 - 0.2 = -0.2.$$

Step 2:

$$\tilde{W}_1 = -0.2 - 0.9(0.2) = -0.38, \quad g_1 = 2(-0.38) + 2 = 1.24,$$

$$v_2 = 0.9(0.2) + 0.1(1.24) = 0.304, \quad W_2 = -0.2 - 0.304 = -0.504.$$

Step 3:

$$\tilde{W}_2 = -0.504 - 0.9(0.304) = -0.7776, \quad g_2 = 2(-0.7776) + 2 = 0.4448,$$

$$v_3 = 0.9(0.304) + 0.1(0.4448) = 0.31808, \quad W_3 = -0.504 - 0.31808 = -0.82208.$$

Look-ahead gradients shrink toward zero as the optimum is approached, NAG naturally decelerates near the minimum instead of overshooting.

Momentum vs. NAG

Method	Gradient evaluated at	Behaviour
SGD	W_t	No memory
Momentum	W_t	Velocity from past gradients
NAG	$W_t - \beta v_{t-1}$ (look-ahead)	Anticipates overshoot

Key Insight

Momentum says: *move, then correct.*
NAG says: *peek ahead, then move.*

20.7 AdaGrad: Per-Parameter Adaptive Learning Rates

The Problem AdaGrad Solves

Problem

All optimizers so far use a single global learning rate η for every parameter. This is suboptimal because:

- Parameters that receive large, frequent gradients (dense features) tend to overshoot.
- Parameters that receive small, infrequent gradients (rare features, sparse inputs) barely move, they are effectively undertrained.

A single η cannot be right for both at once.

Solution

AdaGrad (Duchi et al., 2011) assigns each parameter its own effective learning rate, inversely scaled by the square root of the sum of all squared gradients seen so far. Frequently updated parameters get a smaller rate; rarely updated parameters keep a larger rate.

Where This Matters: Sparse Data Example

Imagine a word-embedding model. The word "the" appears in nearly every sentence, its embedding gets updated thousands of times per epoch. The word "tachycardia" appears in a handful of medical documents, its embedding gets updated almost never. A single learning rate either:

- overshoots the common-word embedding (too large), or
- never meaningfully trains the rare-word embedding (too small).

AdaGrad solves this by *accumulating a history* of gradient magnitudes per parameter and using it to modulate the step size automatically.

Core Formulas

AdaGrad update:

$$G_t = G_{t-1} + g_t^2 \qquad \text{(accumulated squared gradient)}$$

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{G_t + \epsilon}}\, g_t$$

where:

- $G_t \in \mathbb{R}^d$ is the element-wise sum of squared gradients up to step t,
- $g_t = \nabla L(W_t)$ is the current gradient,
- η is the global base learning rate (often set to 0.01),
- $\epsilon \approx 10^{-8}$ prevents division by zero.

Effective learning rate per parameter i:

$$\eta_i^{(t)} = \frac{\eta}{\sqrt{G_t^{(i)} + \epsilon}}.$$

As $G_t^{(i)}$ grows, $\eta_i^{(t)}$ shrinks. A parameter that has received many large gradients will automatically receive smaller future steps.

Numerical Example

Numerical Example

Two-parameter model: $W = [W_1, W_2]$, $\eta = 0.1$, $\epsilon = 10^{-8}$, $G_0 = [0, 0]$.
Suppose gradients across three steps are:

$$g^{(1)} = [3, 0.01], \quad g^{(2)} = [3, 0.01], \quad g^{(3)} = [3, 0.01].$$

W_1 receives large gradients (simulating a common feature). W_2 receives tiny gradients (simulating a rare feature).

Step 1:

$$G_1 = [3^2, 0.01^2] = [9,\ 0.0001].$$

$$\Delta W_1 = \frac{0.1}{\sqrt{9}} \cdot 3 = \frac{0.1}{3} \cdot 3 = 0.1.$$

$$\Delta W_2 = \frac{0.1}{\sqrt{0.0001}} \cdot 0.01 = \frac{0.1}{0.01} \cdot 0.01 = 0.1.$$

Both parameters move by 0.1 at first (no history yet).

Step 2:

$$G_2 = [9 + 9,\ 0.0001 + 0.0001] = [18,\ 0.0002].$$

$$\Delta W_1 = \frac{0.1}{\sqrt{18}} \cdot 3 \approx \frac{0.3}{4.243} \approx 0.0707.$$

$$\Delta W_2 = \frac{0.1}{\sqrt{0.0002}} \cdot 0.01 \approx \frac{0.001}{0.01414} \approx 0.0707.$$

Step 3:

$$G_3 = [27,\ 0.0003].$$

$$\Delta W_1 = \frac{0.1}{\sqrt{27}} \cdot 3 \approx \frac{0.3}{5.196} \approx 0.0577.$$

$$\Delta W_2 = \frac{0.1}{\sqrt{0.0003}} \cdot 0.01 \approx \frac{0.001}{0.01732} \approx 0.0577.$$

Observation: although W_1's gradient is 300× larger than W_2's, both parameters take the *same* step $(0.1, 0.0707, 0.0577)$. AdaGrad divides each gradient by its own history, so the rare feature W_2 gets a per-parameter learning rate 300× larger ($\eta/\sqrt{G} = 10$ vs. 0.033 at step 1). Plain SGD with $\eta = 0.1$ would have moved W_2 by only 0.001 per step. Note also that both steps shrink like $1/\sqrt{t}$: the decaying learning rate discussed next.

The Fatal Problem: Learning Rate Monotonically Decreases

Problem

G_t only ever increases, it accumulates squared gradients and never forgets old ones. As training continues, every $\eta_i^{(t)}$ eventually shrinks so close to zero that no parameter moves meaningfully.
Mathematically:

$$G_t^{(i)} = \sum_{k=1}^{t} \left(g_k^{(i)}\right)^2 \to \infty \qquad \Rightarrow \qquad \frac{\eta}{\sqrt{G_t^{(i)}}} \to 0.$$

AdaGrad *kills its own learning rate.* For non-convex deep networks requiring many thousands of steps, training effectively stops long before convergence.

Pros and Cons of AdaGrad

Advantages	Limitations
Per-parameter adaptive learning rate	Learning rate monotonically shrinks to zero
Excellent for sparse features (NLP, RecSys)	Not suitable for deep networks or long training
Eliminates need to hand-tune η per layer	Accumulates all history; no forgetting
Theoretical convergence guarantees (convex)	Sensitive to initial η choice

Key Insight

AdaGrad's legacy. Despite its limitations in deep learning, AdaGrad was revolutionary: it introduced the idea that each parameter deserves its own step size based on its gradient history. RMSProp and Adam are both direct descendants that fix AdaGrad's dying learning rate problem.

20.8 RMSProp: Fixing AdaGrad's Dying Learning Rate

The Problem RMSProp Solves

Problem

AdaGrad accumulates all past squared gradients indefinitely. For deep networks trained over many steps, the accumulated sum G_t grows without bound, driving the effective learning rate to zero. Training stalls before the model has converged.

Solution

RMSProp (Hinton, 2012) replaces AdaGrad's growing cumulative sum with an **exponential moving average (EMA)** of squared gradients. Old gradients are gradually

forgotten; only recent gradient magnitudes influence the current step size. The learning rate never collapses.

Intuition: Why an Exponential Moving Average?

An EMA assigns exponentially decreasing weight to older observations. With decay factor ρ:

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2.$$

If $\rho = 0.9$, the contribution of a gradient from k steps ago is multiplied by 0.9^k. After 22 steps, that contribution is below 10%. After 44 steps, below 1%. The denominator therefore reflects a *recent* estimate of gradient magnitude, not an ever-growing historical sum.

Core Formulas

RMSProp update:

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2$$

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{s_t + \epsilon}}\, g_t$$

where:

- $s_t \in \mathbb{R}^d$ is the EMA of squared gradients (initialized to $s_0 = 0$),
- ρ is the decay rate (typically 0.9 or 0.99),
- η is the global learning rate (typically 0.001),
- $\epsilon \approx 10^{-8}$ prevents division by zero.

The denominator $\sqrt{s_t}$ approximates the root-mean-square of recent gradients, hence the name **RMSProp** (Root Mean Square Propagation).

AdaGrad vs. RMSProp: The Key Difference

Property	AdaGrad	RMSProp
Gradient accumulation	$G_t = G_{t-1} + g_t^2$	$s_t = \rho s_{t-1} + (1-\rho) g_t^2$
Type	Growing sum (all history)	EMA (fading memory)
Learning rate long-term	$\to 0$ (always)	Stabilises around recent magnitude
Good for	Convex, sparse problems	Non-convex deep networks

Numerical Example

Numerical Example

Given: $W_0 = 1.0$, $\eta = 0.1$, $\rho = 0.9$, $\epsilon = 10^{-8}$, $s_0 = 0$.
Gradients: $g_1 = 2$, $g_2 = 2$, $g_3 = 2$ (constant, to isolate the EMA effect).

Step 1:

$$s_1 = 0.9(0) + 0.1(4) = 0.4.$$

$$W_1 = 1.0 - \frac{0.1}{\sqrt{0.4}} \cdot 2 = 1.0 - \frac{0.2}{0.6325} = 1.0 - 0.3162 = 0.6838.$$

Step 2:

$$s_2 = 0.9(0.4) + 0.1(4) = 0.36 + 0.4 = 0.76.$$

$$W_2 = 0.6838 - \frac{0.2}{\sqrt{0.76}} = 0.6838 - \frac{0.2}{0.8718} = 0.6838 - 0.2294 = 0.4544.$$

Step 3:

$$s_3 = 0.9(0.76) + 0.1(4) = 0.684 + 0.4 = 1.084.$$

$$W_3 = 0.4544 - \frac{0.2}{\sqrt{1.084}} = 0.4544 - \frac{0.2}{1.0411} = 0.4544 - 0.1921 = 0.2623.$$

Key observation: s_t grows initially and then stabilises (it converges to $g^2 = 4$ as $t \to \infty$). Once stable, the effective learning rate $\eta/\sqrt{s} = 0.1/2 = 0.05$ stays *constant*. Unlike AdaGrad, the learning rate does not shrink to zero.

Why RMSProp Still Has a Gap

RMSProp fixes the dying learning rate. However:

- It has no momentum term, it adapts step size but does not use velocity.
- The EMA of squared gradients s_t is biased toward zero early in training (because $s_0 = 0$).
- Steps early in training can be unreliably large before s_t has built up.

Adam combines RMSProp's adaptive scaling *with* momentum, and adds bias correction to fix the early-training problem.

Pros and Cons of RMSProp

Advantages	Limitations
Fixes AdaGrad's dying learning rate	No momentum term
Works well in non-stationary settings	Biased EMA early in training
Effective for RNNs (Hinton's original use)	ρ requires tuning
Simple and fast	No bias correction

20.9 Adam: Adaptive Moment Estimation

The Problem Adam Solves

Problem

After RMSProp, two problems remain:

1. **No momentum.** RMSProp adapts step sizes but does not accumulate velocity. It discards the direction smoothing that momentum provides.
2. **Initialisation bias.** Both the first moment estimate (momentum) $m_0 = 0$ and the second moment estimate (variance) $v_0 = 0$ start at zero. Early in training, these are biased toward zero, causing erroneously large or small steps before the estimates have warmed up.

Solution

Adam (Kingma & Ba, 2015) combines:

- **First moment:** an EMA of gradients (like momentum), smooths the direction.
- **Second moment:** an EMA of squared gradients (like RMSProp), adapts the step size.
- **Bias correction:** divides both estimates by $(1 - \beta^t)$ to counteract the zero initialisation during early steps.

Core Formulas

Adam update rules:
Step 1 — update first moment (mean of gradients):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

Step 2 — update second moment (uncentred variance of gradients):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$$

Step 3 — bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Step 4 — weight update:

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\, \hat{m}_t$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 0.001$.

What Each Part Does: Layered Intuition

Component	Formula	Role
First moment m_t	EMA of g_t	Smoothed gradient direction (momentum)
Second moment v_t	EMA of g_t^2	Recent gradient magnitude (RMSProp)
Bias-corrected $\hat{m}_t$	$m_t/(1-\beta_1^t)$	Removes zero-init bias in direction
Bias-corrected $\hat{v}_t$	$v_t/(1-\beta_2^t)$	Removes zero-init bias in magnitude
Effective step	$\eta\,\hat{m}_t/\sqrt{\hat{v}_t}$	Momentum direction, adaptive size

Why Bias Correction Matters

At $t=1$ with $m_0=0$ and $\beta_1=0.9$:

$$m_1 = 0.9(0) + 0.1\,g_1 = 0.1\,g_1.$$

This underestimates the true gradient by a factor of 10. Without correction, the first step would be ten times too small. The bias-corrected version:

$$\hat{m}_1 = \frac{0.1\,g_1}{1-0.9^1} = \frac{0.1\,g_1}{0.1} = g_1.$$

This restores the correct magnitude at step 1. As t grows, $1-\beta_1^t \to 1$ and the correction vanishes, after warm-up it has no effect.

Numerical Example

Numerical Example

Given: $W_0=0$, $\eta=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$, $m_0=0$, $v_0=0$.
Gradients: $g_1=0.5$, $g_2=0.4$, $g_3=0.6$.

Step 1 ($t=1$)**:**

$$m_1 = 0.9(0) + 0.1(0.5) = 0.05.$$

$$v_1 = 0.999(0) + 0.001(0.25) = 0.00025.$$

$$\hat{m}_1 = \frac{0.05}{1-0.9} = \frac{0.05}{0.1} = 0.5.$$

$$\hat{v}_1 = \frac{0.00025}{1-0.999} = \frac{0.00025}{0.001} = 0.25.$$

$$W_1 = 0 - \frac{0.001}{\sqrt{0.25}+10^{-8}}(0.5) = 0 - \frac{0.001}{0.5}(0.5) = 0 - 0.001 = -0.001.$$

Step 2 ($t=2$)**:**

$$m_2 = 0.9(0.05) + 0.1(0.4) = 0.045 + 0.04 = 0.085.$$

$$v_2 = 0.999(0.00025) + 0.001(0.16) = 0.00024975 + 0.00016 = 0.00040975.$$

$$\hat{m}_2 = \frac{0.085}{1-0.81} = \frac{0.085}{0.19} \approx 0.4474.$$

$$\hat{v}_2 = \frac{0.00040975}{1 - 0.999^2} = \frac{0.00040975}{0.001999} \approx 0.2050.$$

$$W_2 = -0.001 - \frac{0.001}{\sqrt{0.2050}}(0.4474) \approx -0.001 - \frac{0.001}{0.4528}(0.4474)$$

$$\approx -0.001 - 0.000988 \approx -0.001988.$$

Key observation: even with varying gradient magnitudes, Adam produces very stable step sizes because $\hat{m}_t$ smooths the direction while $\sqrt{\hat{v}_t}$ normalises the magnitude. The update is approximately $\pm\eta$ per step during stable training.

Why Adam Is the Default Optimizer in Deep Learning

Key Insight

Adam works well out of the box on most architectures because:

- Momentum smooths noisy gradient directions, giving fast convergence in valleys.
- Adaptive scaling handles parameters with wildly different gradient magnitudes, common in deep networks with different layer sizes.
- Bias correction ensures the first few steps are well-behaved, not stalled or explosive.
- The default hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.001$) work well across a huge range of architectures without modification.

Known Limitations of Adam

Problem

1. **Generalisation gap.** Adam often converges to sharp minima that generalise worse than the flatter minima found by SGD with momentum. SGD is sometimes preferred when final test accuracy is the primary goal.
2. **L2 regularisation is not true weight decay in Adam.** When L2 penalty $\lambda\|W\|^2$ is added to the loss, the gradient becomes $g_t + \lambda W_t$. Adam scales this term by $1/\sqrt{\hat{v}_t}$, which means the effective decay rate is not constant, it is modulated by the adaptive scaling. This breaks the intended regularisation effect.

Pros and Cons of Adam

Advantages	Limitations
Combines momentum + adaptive scaling	May converge to sharp, less-generalisable minima
Bias correction stabilises early steps	L2 regularisation is distorted (use AdamW)
Robust defaults; rarely needs tuning	More memory: stores m_t and v_t per parameter
Handles sparse gradients well	Not provably convergent in all non-convex settings
De facto standard for deep learning	Sensitive to ϵ in some architectures

20.10 AdamW: Decoupled Weight Decay

The Problem AdamW Solves

Problem

In Adam, adding L2 regularisation $\lambda\|W\|^2$ to the loss introduces the extra term λW_t into the gradient. Adam then scales this term by the adaptive factor $1/\sqrt{\hat{v}_t}$, which changes from parameter to parameter and from step to step. The result is that the actual weight decay is neither uniform nor predictable, it is entangled with the gradient history. The intended regularisation effect is corrupted.
Formally, with L2-in-loss Adam:

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t - \underbrace{\frac{\eta\,\lambda}{\sqrt{\hat{v}_t} + \epsilon}W_t}_{\text{distorted decay}}.$$

The decay coefficient $\frac{\eta\lambda}{\sqrt{\hat{v}_t}}$ varies per parameter and per step, not constant weight decay.

Solution

AdamW (Loshchilov & Hutter, 2019) **decouples weight decay from the gradient update**. Instead of adding the L2 penalty to the loss (and having it flow through Adam's scaling), AdamW applies weight decay directly to the weights *after* the Adam gradient step, as a separate multiplicative term:

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t - \eta\lambda W_t.$$

Now $\eta\lambda$ is a fixed, uniform decay per step, independent of the gradient history.

Core Formulas

AdamW update rules:
Steps 1–3: identical to Adam (compute m_t, v_t, bias-correct to $\hat{m}_t$, $\hat{v}_t$).
Step 4 — weight update with decoupled decay:

$$W_{t+1} = \underbrace{W_t(1-\eta\lambda)}_{\text{weight decay}} - \underbrace{\frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t}_{\text{Adam gradient step}}$$

The decay term $(1-\eta\lambda)$ multiplies all weights uniformly every step, regardless of gradient history.

Adam vs. AdamW: Side-by-Side

Property	Adam + L2	AdamW
Regularisation placement	In the loss (gradient)	Directly on weights
Decay term per step	$\eta\lambda/\sqrt{\hat{v}_t}$ (varies)	$\eta\lambda$ (constant)
Gradient scaling affects decay?	Yes (distorted)	No (decoupled)
Effective regularisation	Inconsistent	Clean, uniform
Recommended for	Legacy code	All modern use

Numerical Example

Numerical Example

Setup: $W_0 = 1.0$, $\eta = 0.001$, $\lambda = 0.01$, $\hat{m}_1 = 0.5$, $\hat{v}_1 = 0.25$ (same as Adam Step 1 above).

Adam + L2 in loss (gradient includes λW):

$$\tilde{g}_1 = g_1 + \lambda W_0 = 0.5 + 0.01(1.0) = 0.51.$$

Adam uses $\tilde{g}_1$ to update m_t and v_t, so the decay gets scaled by $1/\sqrt{\hat{v}_t}$, not constant.

AdamW (decay is separate):

$$\begin{aligned} W_1 &= W_0(1 - 0.001 \times 0.01) - \frac{0.001}{\sqrt{0.25}}(0.5) \\ &= 1.0(0.99999) - \frac{0.001}{0.5}(0.5) = 0.99999 - 0.001 = 0.99899. \end{aligned}$$

Decay contribution: $W_0 \times \eta\lambda = 1.0 \times 10^{-5}$ (tiny but constant per step regardless of $\hat{v}_t$).
For parameters with very small $\hat{v}_t$ (small recent gradients), Adam+L2 would apply a huge effective decay; AdamW applies the same $\eta\lambda$ regardless.

Why AdamW Is Now the Recommended Default

AdamW is used in virtually all modern large-scale models (GPT, BERT, ViT, LLaMA, etc.) because:

- Weight decay is interpretable and consistent across all parameters.
- Better generalisation than Adam on most benchmarks.
- PyTorch implements it natively: `torch.optim.AdamW`.
- Drop-in replacement for Adam with only the λ hyperparameter added.

Pros and Cons of AdamW

Advantages	Limitations
Correct, decoupled weight decay	Still inherits Adam's sharp-minima tendency
Better generalisation than Adam	One extra hyperparameter λ
State-of-the-art across most tasks	Same memory cost as Adam
Clean theoretical interpretation	Needs λ tuning (typically 10^{-4} to 10^{-2})

20.11 The Full Evolution: Why Each Optimizer Exists

Each optimizer in this document was created to fix a specific, well-defined failure of its predecessor:

Optimizer	Problem fixed	Mechanism	New limitation
GD	– (baseline)	Full-data gradient	Slow; one update/epoch
SGD	Cost of GD	One-sample gradient	High variance, zig-zag
Mini-batch SGD	Variance of SGD	Batch average gradient	No adaptive step size
Momentum	Zig-zag of SGD	Velocity memory	Reactive; overshoots
NAG	Overshoot of momentum	Look-ahead gradient	Still one global η
AdaGrad	One-size-fits-all η	Accumulated gradient scaling	$\eta \to 0$; dies
RMSProp	Dying η of AdaGrad	EMA of squared gradients	No momentum; biased early
Adam	No momentum in RMSProp; bias	Momentum + EMA scaling + bias correction	Distorted L2 decay
AdamW	Distorted L2 in Adam	Decoupled weight decay	Inherits Adam's sharp minima

20.12 Practical Guidelines: Which Optimizer to Use

Situation	Recommended optimizer
Default starting point	**AdamW** with $\lambda = 10^{-4}$–10^{-2}, $\eta = 10^{-3}$
Best test accuracy (image classification)	**SGD + Momentum** with learning rate schedule
Sparse features (NLP embedding layers)	**AdaGrad** or **Adam**
RNNs / LSTMs	**RMSProp** (Hinton's original recommendation)
Transformers (GPT, BERT, ViT)	**AdamW** with cosine LR schedule
Research / reproducibility	Document all hyperparameters; default to AdamW

Key Insight

The practitioner's rule of thumb:

- Start with **AdamW**. It works reliably with minimal tuning on almost any architecture.
- If you need the last fraction of accuracy on a vision benchmark, try **SGD + Momentum** with a tuned cosine or step learning-rate schedule.
- Avoid vanilla **Adam** in new code — use AdamW instead. The weight decay is always more interpretable.
- **AdaGrad** is rarely used in deep learning today but remains the right choice for online/sparse learning problems.

20.13 Summary: All Formulas in One Place

SGD:

$$W_{t+1} = W_t - \eta\, g_t$$

Momentum:

$$v_t = \beta v_{t-1} + (1-\beta) g_t, \qquad W_{t+1} = W_t - \eta v_t$$

NAG:

$$\tilde{W}_t = W_t - \beta v_{t-1}, \qquad g_t = \nabla L(\tilde{W}_t), \qquad v_t = \beta v_{t-1} + \eta g_t, \qquad W_{t+1} = W_t - v_t$$

AdaGrad:

$$G_t = G_{t-1} + g_t^2, \qquad W_{t+1} = W_t - \frac{\eta}{\sqrt{G_t + \epsilon}}\, g_t$$

RMSProp:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad W_{t+1} = W_t - \frac{\eta}{\sqrt{s_t + \epsilon}}\, g_t$$

Adam:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t$$

AdamW:

$$W_{t+1} = W_t(1-\eta\lambda) - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t$$

21 Non-Technical Guide: What Every Concept Really Means

---

1. Dropout — "Don't let the team get lazy"

Imagine you are training a football squad of 100 players. If the same 11 always play, the other 89 never improve and the starting 11 learn to cover for each other's weaknesses in very specific, rigid ways. The moment you change the lineup, say, one player is injured, the team collapses because no one else knows the role.

A neural network has the same problem. With millions of neurons firing together on every example, groups of neurons learn to rely on each other in hard-wired ways. Neuron A always corrects for Neuron B's mistakes. Remove A, and the prediction falls apart. The network has *memorised* the training data rather than learning anything general.

Dropout fixes this by telling the coach: before every single practice session, randomly sit out $p\%$ of the squad. Today you lose neurons 3, 7, 19, and 42. Tomorrow you lose a completely different set. Every neuron must learn to do its job *regardless* of who else is on the field. Over thousands of practice sessions, you end up with a squad where every player is independently capable.

At match day (test time) you field the entire squad at once. Because every player was trained to work without their usual teammates, having everyone together gives you a kind of superteam, an implicit average of thousands of different training lineups. No random dropout happens during the actual game.

The scaling trick (dividing the surviving neurons' outputs by $1-p$ during practice) makes up for the missing players, so the average total "signal" in practice already matches what the full squad produces. On match day everyone plays and nothing needs rescaling.

One sentence: Dropout randomly silences neurons during training so no neuron can free-ride, forcing the whole network to become robust and independently capable.

---

2. Batch Normalisation — "Serve every dish at the same temperature"

Picture a restaurant kitchen with many cooking stations in sequence. Station 1 prepares the ingredients and passes them to Station 2, which cooks them and passes to Station 3, and so on. The problem: every time the chef at Station 1 tweaks her recipe, the temperature, texture, and seasoning of what she hands to Station 2 changes. Station 2 then has to re-learn how to handle the new input, and when Station 2 changes, Station 3 has to re-learn

again, and so on down the line. The entire kitchen is constantly re-adjusting. Progress is slow.

In a neural network, this is called *internal covariate shift*. As earlier layers update their weights during training, the distribution of signals they send to later layers keeps changing. Later layers are always chasing a moving target. They spend most of their effort re-adapting to upstream changes rather than learning the actual task.

Batch Normalisation is like installing a standardisation counter between each station: before passing food to the next station, rescale everything so it always arrives at roughly the same temperature and portion size (mean ≈ 0, variance ≈ 1). Now Station 2 always receives a predictable input and can focus entirely on learning its own job well.

The learnable scale (γ) and shift (β) parameters added after normalisation give the network the freedom to say: "Actually, for this particular layer, the optimal input is not mean-zero, let me shift and rescale it to whatever works best." The network normalises first and then optionally undoes the normalisation to whatever distribution the next layer needs.

One sentence: Batch Normalisation standardises the signals flowing between layers so each layer always sees a stable, predictable input, letting the whole network train faster and more reliably.

---

3. Weight Initialisation — "Start at the right volume before you turn on the music"

Before a network trains a single example, every weight must be given a starting value. You might think this does not matter much, after all, the weights will be updated thousands of times. But the starting values can doom training entirely before it even begins.

Think of a sound mixing board. Before the show starts, if every fader is pushed to maximum, the speakers will immediately blow. If every fader is at zero, no sound comes out at all. You need to start with every fader at a sensible middle level so the first note is audible, undistorted, and in the right range for the performers to respond to.

A neural network has the same constraint. If all weights start too large, the signal amplifies at every layer, after 20 layers it is astronomically large, and the gradients explode (NaN values, crashed training). If all weights start too small, the signal shrinks at every layer, after 20 layers it is essentially zero, and the gradients vanish (the network cannot learn anything, because the learning signal has disappeared).

Xavier initialisation (for tanh and sigmoid) sets each starting weight to a carefully chosen small random value based on the number of inputs and outputs a neuron has. He initialisation does the same for ReLU networks, but uses a slightly larger range to compensate for the fact that ReLU kills half of all signals by mapping negatives to zero.

Neither scheme is magic, they are just careful answers to the question: what range of starting values ensures that, at the very first forward pass, the signal neither explodes nor vanishes?

One sentence: Weight initialisation sets the starting dial positions of the network so that signals flow through all layers at a healthy magnitude from the very first step.

---

4. Optimizers — "How to navigate a foggy mountain landscape at night"

Imagine you are blindfolded on a hilly landscape and your goal is to reach the lowest valley. You cannot see the whole terrain, you can only feel the slope under your feet right now. Your strategy for deciding which direction to step next, and how big each step should be, is your *optimizer*.

Gradient Descent is like measuring the exact slope under your feet by sampling every single point on the map before taking one step. The direction is perfect, but it takes forever to compute, you cannot afford it on a large landscape.

SGD is like closing your eyes, poking one random spot of ground, and immediately stepping in that direction. It is fast but noisy. Sometimes you step the wrong way entirely because one poke was misleading.

Mini-batch SGD is the sensible middle ground. Poke a small patch of ground (say, 32 nearby points), average the slope you feel, then step. Much faster than measuring the whole map, much less misleading than a single poke.

Momentum is like being on a sledge. Instead of reacting to every individual bump and changing direction instantly, you keep moving in roughly the direction you have been going, only gradually adjusting. This smooths out the left-right zigzag and lets you glide steadily toward the valley.

NAG (Nesterov) is like being on that same sledge but leaning your body forward to look ahead before you apply the brakes. Instead of measuring the slope where you currently are, you first imagine sliding a little further in your current direction, then measure the slope *there*. This gives you earlier warning that a valley floor is approaching, so you slow down before you overshoot.

AdaGrad notices that different parts of the landscape have been visited at very different frequencies. Well-worn paths (frequent, large gradients) get shorter steps, you are probably close to something good there. Rarely visited cliff edges (sparse, small gradients) get longer steps, you need to explore further. The problem: AdaGrad never forgets any old terrain, so the "memory" of every path ever taken accumulates until all steps become infinitesimally small. The sledge grinds to a halt.

RMSProp fixes this by giving AdaGrad a fading memory. Recent slopes matter; slopes from 1000 steps ago barely influence you anymore. The adaptive step-size logic is preserved, but the accumulation that was killing the learning rate is replaced with a gentle forgetting curve.

Adam combines the sledge momentum of Momentum with the adaptive footing of RMSProp, and adds one more fix: in the very first steps, both the momentum estimate and the terrain estimate start at zero (cold), which would make the first few steps erratic. Adam corrects for this cold-start bias so the very first steps are as well-calibrated as steps taken later.

AdamW is Adam with one additional fix. When you tell Adam "penalise large steps" (weight decay), the adaptive scaling corrupts the penalty, some parameters get penalised more than others in an unpredictable way. AdamW separates the penalty entirely from the adaptive update so that every parameter shrinks by the same, clean, predictable fraction each step.

One sentence: Optimizers are navigation strategies for a foggy landscape, each successive one adds smarter memory, better footing, or a cleaner look-ahead to the previous strategy's limitations.

---

5. Regularisation — "Prefer simple answers"

A model that memorises its training examples is like a student who only studied the exact past exam papers. They can reproduce every answer from those papers perfectly, but the moment a question is phrased slightly differently, they are lost. They have not learned the subject, they have learned the specific questions.

Regularisation is the teacher saying: "I will subtract marks if your answer is unnecessarily complicated." This forces the student to look for the simplest explanation that still fits all the examples rather than the most elaborate memorisation strategy.

L2 regularisation (weight decay) says: "Every weight in the network has a small running cost. The bigger the weight, the bigger the cost." The network therefore avoids making any single weight large unless the data genuinely justifies it. Large weights create sharp, complicated decision boundaries. Small weights create smooth, gentle ones. The result is a model that generalises.

L1 regularisation charges in proportion to the size of each weight, $|w|$, rather than its square. The consequence is dramatic: the pull toward zero is the same strength however small the weight already is, so weights that are only weakly useful are driven all the way to exactly zero. L1 therefore performs automatic feature selection: the model decides which inputs are worth paying attention to at all, and switches off the rest entirely.

Think of L2 as a tax that grows with the square of your size: big weights pay a lot, small ones almost nothing, so they shrink but rarely vanish. L1 is a tax with a constant rate per unit of size: even the smallest weight is still pushed toward zero at full strength, so small weights get eliminated.

One sentence: Regularisation adds a cost for complexity, steering the network toward simpler explanations that generalise better than memorised ones.

---

6. How Everything Fits Together

These five tools work at different stages and solve different problems. They are not alternatives, they work together:

Tool	When it acts	What problem it prevents
Weight Init	Before step 1	Signals exploding or vanishing from the very first pass
Batch Norm	Every forward pass	Distribution shift between layers slowing training down
Dropout	Every training pass	Neurons memorising and relying on each other
Regularisation	Every weight update	Weights growing large and creating overly complex boundaries
Optimizers	Every weight update	Getting stuck, zigzagging, or taking poorly-sized steps

The Big Picture in Plain English

Training a neural network is like teaching a team of thousands of specialists to collaborate on an enormous puzzle, in a dark room, using only indirect feedback (the loss value) about whether their current arrangement is right.
Initialisation makes sure no one starts by screaming (exploding) or whispering (vanishing) before the puzzle has even begun.
Batch Normalisation ensures the signals passed between specialists always arrive at a sensible volume, so no one has to keep readjusting just because an upstream colleague changed their approach.
Dropout stops cliques from forming, forces every specialist to be competent on their own rather than dependent on a fixed partner.
Regularisation stops specialists from memorising irrelevant quirks of the training examples by taxing unnecessary complexity.
Optimizers decide, at each moment, how confidently and in which direction the whole team should move, using the gradient as the compass and learned history to avoid walking in circles.
All five are needed. Remove any one of them and a different failure mode takes over.

22 Evaluating a Classifier

22.1 Confusion Matrix and Threshold Metrics

A binary classifier outputs a probability $\hat{y}$; a **threshold** t (default 0.5) turns it into a decision. On held-out data, count the four outcomes:

	Predicted positive	**Predicted negative**
Actually positive	TP (true positive)	FN (false negative, *missed*)
Actually negative	FP (false positive, *false alarm*)	TN (true negative)

$$\text{Accuracy} = \frac{TP + TN}{\text{all}}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \qquad \text{Specificity} = \frac{TN}{TN + FP}$$

Numerical example (100 test patients): $TP = 40$, $FP = 10$, $FN = 8$, $TN = 42$.

$$\text{Accuracy} = \tfrac{82}{100} = 0.82, \qquad \text{Precision} = \tfrac{40}{50} = 0.80, \qquad \text{Recall} = \tfrac{40}{48} = 0.833$$

$$F_1 = \tfrac{2(0.80)(0.833)}{0.80+0.833} = 0.816, \qquad \text{Specificity} = \tfrac{42}{52} = 0.808$$

In Plain Words

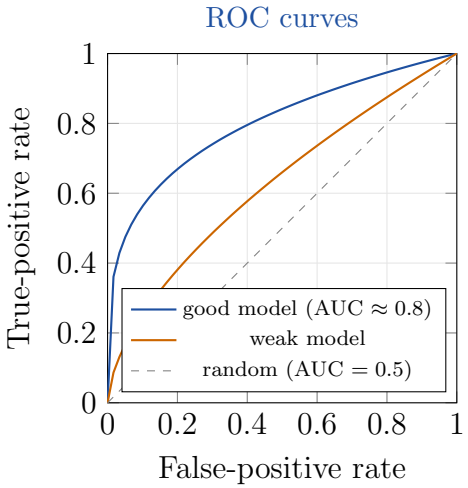
Precision: of the patients we flagged, how many were actually sick? **Recall**: of the sick patients, how many did we catch? Lowering the threshold catches more sick patients (higher recall) at the cost of more false alarms (lower precision). Choose the threshold by the cost of each mistake: in disease screening a miss is expensive, so favour recall; for fraud alerts sent to customers, false alarms annoy people, so favour precision.

22.2 ROC Curve and AUC

The ROC curve plots the true-positive rate (recall) against the false-positive rate $FP/(FP + TN)$ as the threshold sweeps from 1 to 0. The area under it has a direct meaning:

$$\text{AUC} = P\big(\hat{y}(\mathbf{x}^+) > \hat{y}(\mathbf{x}^-)\big)$$

the probability that a randomly chosen positive is scored higher than a randomly chosen negative. AUC = 0.5 is random guessing, 1.0 is perfect ranking. It does not depend on the threshold.



Note

For rare positives (fraud at 0.1%), accuracy and even ROC-AUC can look excellent while the model is useless in practice. Report precision, recall and the area under the precision–recall curve (PR-AUC) instead.

22.3 Regression Metrics

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}, \qquad R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

MAE and RMSE are in the units of the target (e.g. dollars); RMSE punishes large errors more. R^2 is the fraction of variance explained (1 is perfect, 0 is no better than predicting the mean).

23 Putting It All Together: An MLP in PyTorch

Every line below corresponds to a concept in these notes.

Binary classifier: 13 inputs → 16 → 12 → 8 → 4 → 1

```
import torch, torch.nn as nn

def block(n_in, n_out, p=0.3):   # Linear-BN-ReLU-Dropout
    return [nn.Linear(n_in, n_out), nn.BatchNorm1d(n_out),
            nn.ReLU(), nn.Dropout(p)]

model = nn.Sequential(*block(13, 16), *block(16, 12), *block(12, 8),
                      *block(8, 4), nn.Linear(4, 1))  # logit output

for m in model:                              # He init for ReLU layers
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight)
        nn.init.zeros_(m.bias)

loss_fn = nn.BCEWithLogitsLoss()   # sigmoid + BCE, stable
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

best, patience = float("inf"), 0
for epoch in range(50):
    model.train()                            # BN uses batch stats, dropout on
    for xb, yb in train_loader:              # mini-batches (e.g. 32)
        opt.zero_grad()                      # clear old gradients
        loss = loss_fn(model(xb).squeeze(1), yb)  # forward+loss
        loss.backward()                      # backpropagation
        opt.step()                           # weight update
    sched.step()                             # learning-rate schedule

    model.eval()                             # BN running stats, dropout off
    with torch.no_grad():
        val = sum(loss_fn(model(xb).squeeze(1), yb).item()
                  for xb, yb in val_loader) / len(val_loader)
    if val < best:                           # early stopping
        best, patience = val, 0
        torch.save(model.state_dict(), "best.pt")
    else:
        patience += 1
        if patience >= 10: break
```

23.1 Practical Checklist

1. Split train / validation / test (by time if the data has a time order). Fit scalers on training data only.
2. Fit a baseline first (logistic regression or gradient boosting).
3. Choose the output layer and loss from the task (Section on output layers).
4. Hidden layers: ReLU, He initialization, batch norm; start with 2–4 layers.
5. Optimizer: AdamW, $\eta = 10^{-3}$; add a cosine schedule.
6. Regularize: dropout 0.1–0.3, weight decay 10^{-4}–10^{-2}, early stopping.
7. Plot training and validation loss every run; diagnose under- or over-fitting.
8. Choose the decision threshold by business cost; evaluate the test set once.

End of Chapter 1

Multi-Layered Perceptron — Complete Notes